

THE BEGINNER'S GUIDEBOOK

LLM Powered AI Chatbot

Understand, run, change and deploy a real streaming chatbot built with React, Django and Google Gemini. No prior experience with the project needed.

React 19

Vite

Tailwind CSS

Django 6

Django REST Framework

PostgreSQL

Redis

Google Gemini

Server-Sent Events

Explain Python decorators

ASSISTANT

A decorator is a function that wraps another function to add behaviour

```
@timed def slow_add(a, b):
```

● ● ● streaming...

BEFORE YOU START

About This Guide

This book walks you through one real, working application from the outside in: what it does, how to use it, how to run it on your own computer, how every piece fits together, and how to change it safely.

Who this guide is for

You don't need to be an expert. If you can open a terminal and type commands, have seen a little code in any language, and are curious how a chatbot like ChatGPT is built behind the scenes, this guide is for you. If you already know Django or React, skim Part I and go straight to Part III, where the project-specific details live.

Pick your reading path

If you want to...	Read
Just use the chatbot	Chapter 2 (A Tour of the App)
Understand the ideas before touching code	Chapters 1, 3 and 4
Run the project on your own computer	Chapters 5 and 6, then 17 if something breaks
Learn how the code works	Chapters 7 to 11
Make your first change	Chapters 13 and 14
Put it on the internet	Chapters 15 and 16

Conventions used in this book

A **dark box** is something you type into a terminal. Lines starting with `#` are comments; you don't type them.

Terminal

```
# Which Python do I have?
python3 --version
```

A **light box** shows the contents of a file. The file's path is in the box's header.

backend/chat/services/prompts.py

```
SYSTEM_PROMPT_VERSION = "1.0.0"
```

File paths like `backend/chat/views.py` are always relative to the project's root folder. Coloured boxes mark different kinds of advice:

TIP

A shortcut or good habit.

NOTE

Useful background.

WATCH OUT

A common mistake.

IN THIS PROJECT

Where an idea shows up in the code.

Everything here was checked against the source code at commit `2821ab9`. Where the code and the older docs disagree, this guide follows the code and says so. If the code has moved on since, the files named in each section are the source of truth.

What's Inside

PART I · MEET THE PROJECT

1	What This Project Is	5
	The problem, the features, the three big pieces	
2	A Tour of the App	7
	Signing in, chatting, the sidebar, themes, errors	
3	Core Ideas in Plain Language	11
	APIs, HTTP, LLMs, tokens, streaming, JWT, databases	
4	The Technology Stack	14
	Every tool, what it does and where it lives	

PART II · GET IT RUNNING

5	Preparing Your Computer	17
	Installing Python, Node, PostgreSQL, Redis; getting a Gemini key	
6	Running the Project Locally	19
	Step by step, plus the Docker shortcut	
7	A Map of the Codebase	24
	Every folder, and where to look for what	

PART III · HOW IT WORKS

8	The Journey of One Message	27
	From your keyboard to Gemini and back, step by step	
9	Inside the Backend	30
	Django apps, the chat endpoint, the Gemini service, errors, limits	
10	Inside the Frontend	35
	Routing, auth, reading a stream, the chat state machine	
11	The Data Model	38
	Users, conversations, messages and usage records	

PART IV · BUILD, TEST, SHIP

12	Configuration Reference	41
	Every environment variable explained	
13	Testing	43
	Running the suites, what they cover, writing your first test	
14	Your First Changes	46
	Five guided exercises	
15	Deployment	48
	Vercel, the VPS, nginx and the CI/CD pipeline	
16	Security Essentials	51
	What protects users, and the rules you must not break	
17	Troubleshooting	53
	Symptoms, causes and fixes	

APPENDICES

A	Glossary	55
B	Command Cheat Sheet	57
C	API Quick Reference	58
D	Design Decisions and Further Reading	60

PART I

Meet the Project

What the chatbot does, how people use it, and the handful of ideas you need before the code will make sense.

-
- 1 What This Project Is
 - 2 A Tour of the App
 - 3 Core Ideas in Plain Language
 - 4 The Technology Stack
-

PART I · CHAPTER 1

What This Project Is

A web chatbot, similar in spirit to ChatGPT, that answers questions using Google's Gemini AI model. You type a message, and the answer appears word by word, formatted with headings, lists, tables and highlighted code.

The problem it solves

Calling an AI model is easy. Building a *product* around one is harder. A real chatbot has to remember conversations, keep each user's chats private, show the answer as it is being written, cope when the AI service is slow or down, stop one user from running up a huge bill, and never leak the secret key that pays for the AI. This project does all of that, and it was built to a written specification

([docs/LLM_Powered_AI_Chatbot_PRD.pdf](#), the "PRD", or Product Requirements Document). The code often mentions PRD section numbers in comments, like (PRD 5.3), so you can trace every feature back to the requirement that asked for it.

What it can do

- **Accounts.** Register, sign in and sign out. Your session survives a page reload.
- **Streaming replies.** The answer appears token by token, not all at once after a long wait.
- **Memory.** Follow-up questions keep the context of earlier messages in the same conversation.
- **Rich formatting.** Markdown headings, lists, tables and links; code blocks with syntax highlighting and a Copy button.
- **Conversation history.** A sidebar lists your chats, grouped by date. Rename or delete them.
- **Automatic titles.** A new conversation gets a short title written by the AI.
- **Honest errors.** Every failure shows a clear message and, where it makes sense, a Retry button.
- **Light and dark themes,** a mobile layout, keyboard and screen-reader support.

The three big pieces

Almost every modern web app is split into parts that run in different places. This one has three, plus two helpers:

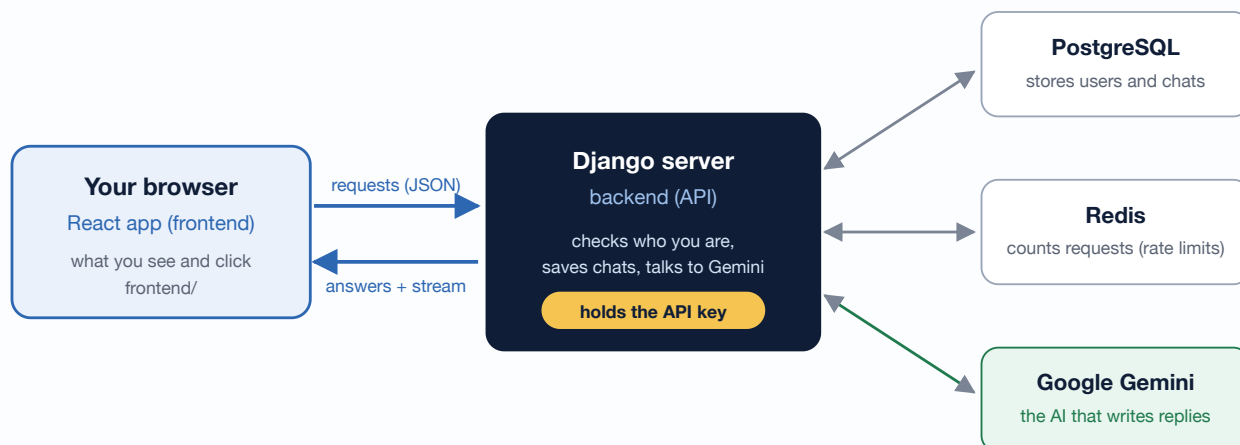


Figure 1.1 The browser only ever talks to Django. Django talks to the database, Redis and Gemini.

Piece	Lives in	Job
Frontend	<code>frontend/</code>	The screens you see: sign-in page, chat window, sidebar. Written in React. Runs inside your web browser.
Backend	<code>backend/</code>	The server. Checks who you are, stores conversations, sends your message to Gemini and relays the answer back. Written in Python with Django.
AI model	Google's servers	Gemini reads the conversation and writes the reply. The project reaches it over the internet with a secret API key.
Database	PostgreSQL	Permanent storage: users, conversations, messages, usage records.
Cache	Redis	Fast, temporary storage. Used to count requests so nobody can send too many messages too quickly.

WHY NOT CALL GEMINI STRAIGHT FROM THE BROWSER?

Anything that runs in a browser can be read by the person using it. If the React app held the Gemini API key, anyone could open their browser's developer tools, copy the key, and spend your money. So the key lives only on the server, and the browser never talks to Google directly. This is the single most important design rule in the project.

Where it runs today

Live app	https://chatbot.technicalbind.com (the frontend, hosted on Vercel)
Live API	https://chatbot.devtechguru.cloud (the backend, on a Linux server)
Source code	github.com/tausif1337/llm-powered-ai-chatbot

The top-level [README.md](#) still mentions an older `vercel.app` address and a Render setup. Those are out of date; [docs/DEPLOYMENT.md](#) describes the current setup, and so does Chapter 15.

PART I • CHAPTER 2

A Tour of the App

Before reading any code, use the app. Knowing what each button does makes the code much easier to follow later. The screenshots in this chapter come from the project running locally.

Signing in

AI Chatbot

Sign in

Continue to your conversations.

Try it without signing up [Use demo](#)

email demo@devtechguru.cloud
pass TryTheChatbot!2026
Shared account — anyone can see these conversations.

Email

you@example.com

Password

.....

Sign in

No account? [Create one](#)

Figure 2.1 The sign-in page, with the optional demo-account panel at the top.

Enter your email and password and press **Sign in**. No account yet? Click **Create one** to open the registration page. It asks for:

- **Email:** this is your username. Capital letters are ignored, so `Sam@example.com` and `sam@example.com` are the same account.
- **Display name** (optional): shown in the sidebar.
- **Password:** at least 10 characters. It also can't be a well-known password (like `password123`), all numbers, or too similar to your email. If the server rejects it, the reason appears under the field.

THE DEMO ACCOUNT IS SHARED

The **Use demo** button signs you in to a public account (`demo@devtechguru.cloud`). Everyone who clicks it sees the same conversations, so never type anything private there. The panel only appears when a deployment switches it on (see Chapter 6).

Once you're signed in, the app remembers you. Close the tab, come back tomorrow, and you're still signed in, for up to seven days of inactivity. Chapter 3 explains how.

Starting a conversation

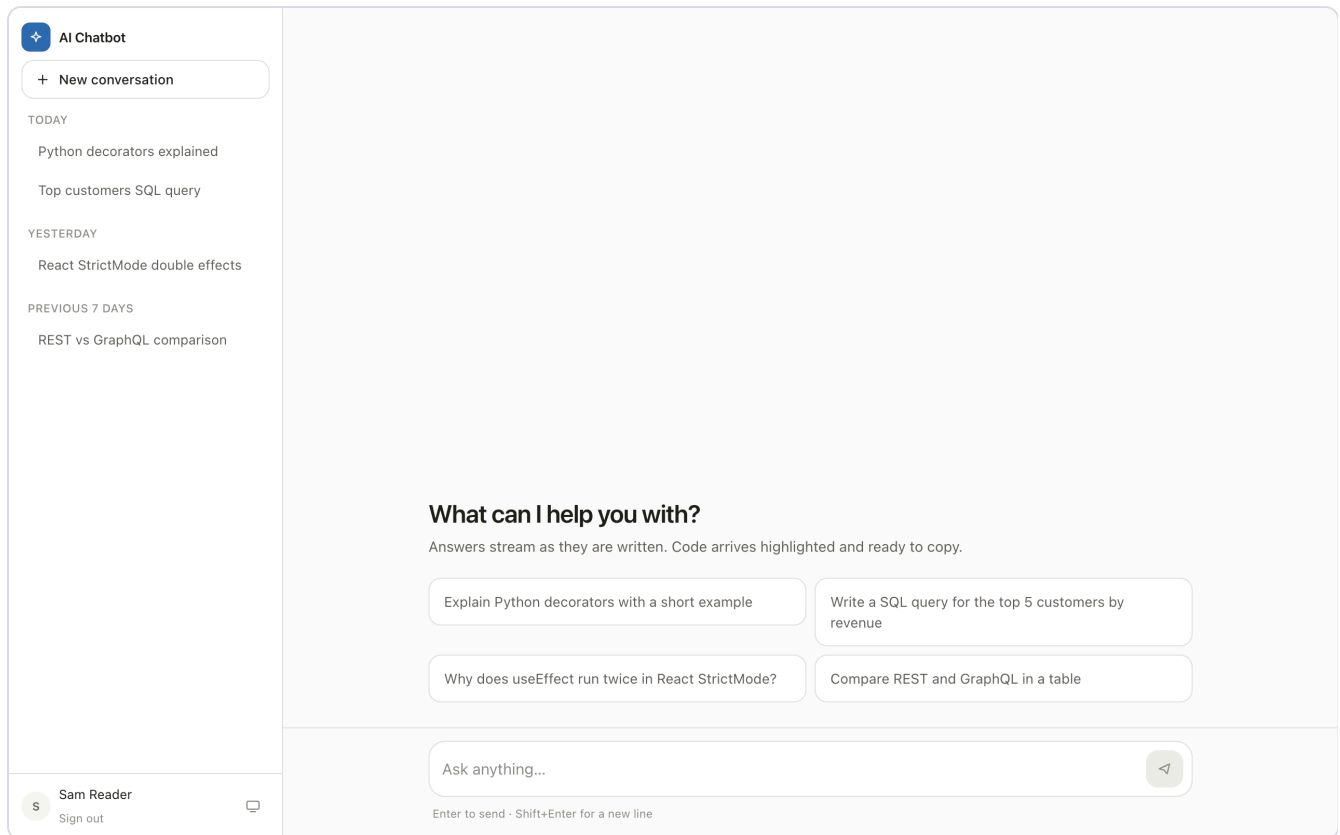


Figure 2.2 A new conversation. The four suggestion buttons send a ready-made question.

Type in the box at the bottom ("Ask anything...") and press **Enter** to send. To start a new line without sending, press **Shift** + **Enter**. You can also click one of the suggestion buttons.

What happens next, in order:

- 1 Your message appears on the right straight away.
- 2 A **"Thinking"** indicator with three pulsing dots appears. Gemini usually takes about three seconds before it sends the first word.
- 3 The answer streams in, with a blinking caret at the end, until it's complete.
- 4 For a brand-new conversation, the sidebar entry changes from "New conversation" to a short AI-written title.

While an answer is streaming, the Send button turns into a **Stop** button. Press it to stop the reply early. The text you already have stays on screen, marked "Stopped."

Reading and reusing answers

Figure 2.3 shows a finished answer. Around it:

- **Copy a code block:** click **Copy** in the code block's header. It changes to "Copied" for two seconds.
- **Copy a whole answer:** hover over the answer and a **Copy** button appears underneath. For a reply you've just received, its token count shows there too.
- **Long code lines** scroll sideways inside the block; a fade on the right edge tells you there's more.
- **Scrolling up** while an answer streams stops the auto-scroll, so you can read in peace. A **Jump to latest** button takes you back down.
- Messages can be up to **32,000 characters**. A counter appears once you pass 80% of that.

AI Chatbot

+ New conversation

TODAY

Python decorators explained

Top customers SQL query

YESTERDAY

React StrictMode double effects

PREVIOUS 7 DAYS

REST vs GraphQL comparison

Sam Reader
Sign out

Explain Python decorators with a short example

ASSISTANT

A **decorator** is a function that takes another function and returns a new one that adds behaviour, without changing the original code.

```
python
import time

def timed(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter() - start:.3f}s")
        return result
    return wrapper

@timed
def slow_add(a, b):
    time.sleep(0.5)
    return a + b

slow_add(2, 3)  # prints: slow_add took 0.501s
```

What `@timed` does:

1. Python calls `timed(slow_add)`.
2. The returned `wrapper` replaces `slow_add`.
3. Every call now runs the timing code around the original.

Common use	Example
Logging	<code>@log_calls</code>
Caching	<code>@functools.cache</code>
Access control	<code>@login_required</code>

Ask anything...

Enter to send · Shift+Enter for a new line

Figure 2.3 A finished answer: bold text, a highlighted Python code block with its own Copy button, a numbered list and a table.

The sidebar

The sidebar on the left lists your conversations, newest activity first, grouped under *Today*, *Yesterday*, *Previous 7 days* and *Older*. It shows your 25 most recently active conversations.

To...	Do this
Open a conversation	Click its title. The address bar changes to <code>/c/<conversation-id></code> , so you can bookmark it.
Start a new one	Click New conversation at the top.

To...	Do this
Rename	Hover and click the pencil. Type, then press Enter to save or Esc to cancel.
Delete	Hover and click the bin. It changes to Sure? ; click again to confirm.
Change theme	Click the small monitor/sun/moon icon at the bottom. It cycles <i>System</i> → <i>Light</i> → <i>Dark</i> .
Sign out	Click Sign out under your name.

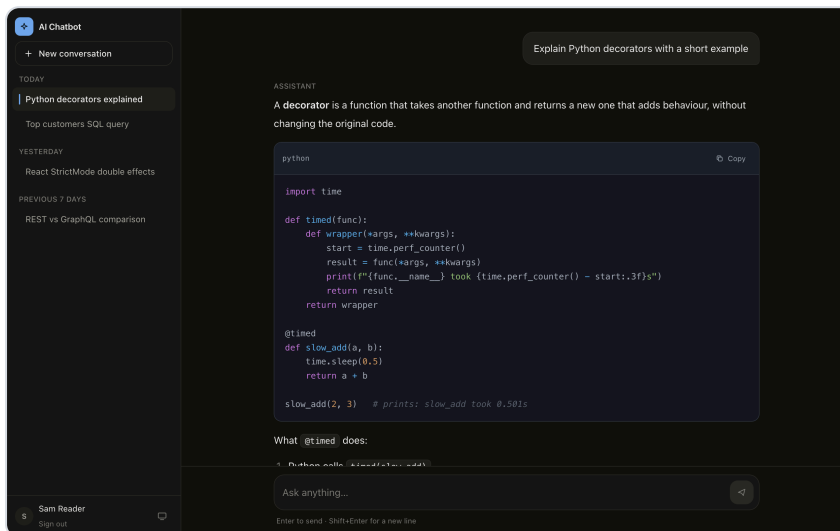


Figure 2.4 Dark theme.

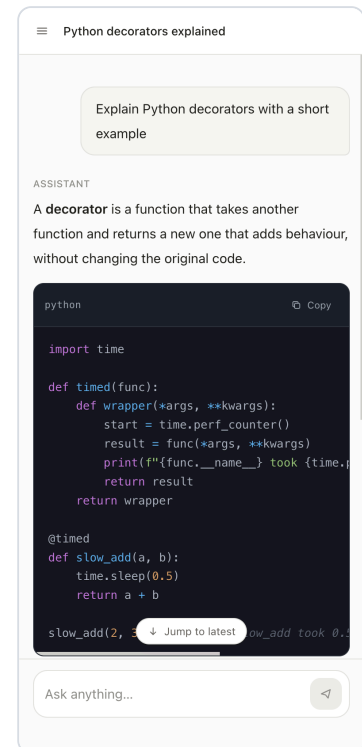


Figure 2.5 On a phone the sidebar hides behind the ≡ menu button.

When something goes wrong

Errors appear as a red banner. Each banner has three parts:

- a plain-English **message**, for example *"The AI service took too long to respond. Please try again."*
- a **Ref** code (like `Ref 7f3a...`). That's the *request ID*. If you report a bug, include it: the server logged the failure under exactly that ID.
- a **Retry** button, but only for problems that retrying can fix (a timeout or a network drop, say). Validation errors, like an empty message, don't get one.

If an answer fails halfway, the text that already arrived stays on screen with a small **incomplete** label, and the error appears beneath it.

PART I · CHAPTER 3

Core Ideas in Plain Language

Ten ideas explain almost everything in this project. Each gets a short explanation in everyday terms, followed by where it shows up in the code.

1. Client and server

Think of a restaurant. You (the **client**) sit at a table and order from a menu. The kitchen (the **server**) prepares the food. You never walk into the kitchen; you make requests and the waiter brings back results.

In web apps, your browser is the client and a computer somewhere on the internet is the server. The **frontend** is the menu and the table: everything you see. The **backend** is the kitchen. The **database** is the pantry where ingredients are kept. And Gemini is a specialist chef the kitchen phones for the hardest dish. Here the frontend is `frontend/` (React) and the backend is `backend/` (Django); they talk only through the API described next.

2. HTTP, requests and status codes

The browser and server talk using **HTTP**. Every conversation is a *request* ("please give me my conversation list") followed by a *response*. A request has a **method** that says what kind of action it is:

Method	Meaning	Example in this project
<code>GET</code>	Read something	<code>GET /api/conversations/</code> lists your chats
<code>POST</code>	Create or do something	<code>POST /api/chat/</code> sends a message
<code>PATCH</code>	Change part of something	<code>PATCH /api/conversations/<id>/</code> renames a chat
<code>DELETE</code>	Remove something	<code>DELETE /api/conversations/<id>/</code> deletes a chat

Every response carries a three-digit **status code**. The first digit tells you the category: `2xx` means success, `4xx` means the request was wrong, and `5xx` means the server (or something it depends on) failed. You'll meet these most often:

<code>200</code> / <code>201</code>	OK / Created	<code>404</code>	Not found, or not yours
<code>204</code>	Done, nothing to send back	<code>413</code>	Too large
<code>400</code>	Invalid input	<code>429</code>	Too many requests, slow down
<code>401</code>	Not signed in, or the token expired	<code>502</code> / <code>504</code>	The AI service failed / timed out

3. JSON and APIs

JSON is a simple text format for data, made of `"name": value` pairs in curly braces. An **API** (Application Programming Interface) is a list of addresses, called **endpoints**, that a program can call, together with the rules for what to send and what comes back. The frontend uses the backend's API; people never type these addresses themselves.

What the browser sends to POST /api/chat/

```
{
  "conversation_id": null,
  "message": "Explain Python decorators with a short example"
}
```

4. Large language models (LLMs) and tokens

An **LLM** is an AI model trained on a huge amount of text. It works by predicting what text should come next, one small piece at a time. Those pieces are called **tokens**. A token is roughly four characters of English, so "chatbot" might be two tokens. Tokens matter for three reasons:

- **Cost.** Google bills per token, both for what you send (*input*) and what the model writes (*output*).
- **Limits.** A model can only read so many tokens at once. That limit is its *context window*.
- **Thinking tokens.** Newer Gemini models "think" privately before answering. Those tokens are billed but never shown.

An LLM has no memory of its own. To "remember" a conversation, the server sends the earlier messages along with every new one. It also sends a **system instruction**: hidden guidance such as "use Markdown, and never invent APIs". **Temperature** controls randomness: 0 is predictable, higher values are more creative. This project uses 0.7.

IN THIS PROJECT

The system instruction lives in `backend/chat/services/prompts.py`. The code that picks which earlier messages to send is `backend/chat/services/context.py`. Token counts are saved on every reply.

5. Streaming and Server-Sent Events (SSE)

A normal request works like ordering a book by post: nothing, nothing, then the whole thing arrives at once. **Streaming** is more like a live broadcast: the answer arrives in pieces as it's being written. With an AI answer that takes 10 to 20 seconds to finish, streaming is what makes the app feel fast.

This project streams with **Server-Sent Events**, a simple standard where the server keeps the response open and writes small labelled messages called *events* (or *frames*). Each frame ends with a blank line:

What the stream looks like on the wire

```
event: meta
data: {"conversation_id": "6b37a2a1-...", "model": "gemini-3.6-flash", ...}

event: delta
data: {"text": "A decorator is a function"}

: heartbeat

event: done
data: {"message_id": "...", "title": "Python decorators explained", "usage": {...}}
```

`meta` opens the stream, each `delta` carries the next piece of text, and `done` (or `error`) closes it. A line starting with a colon is a comment. The server sends a `: heartbeat` during long pauses so that proxies between you and the server don't decide the connection has died.

6. Databases, the ORM and migrations

A **database** stores data in tables, like spreadsheets with strict columns. PostgreSQL is the database used here. Instead of writing SQL by hand, Django lets you describe each table as a Python class, called a **model**, and translates your Python into SQL for you. That translation layer is the **ORM** (Object-Relational Mapper).

When a model changes, the database must change too. A **migration** is a generated file describing one such change; `make migrate` applies any your database hasn't seen yet.

7. Cache and rate limiting (Redis)

Redis is a very fast in-memory store. Here it counts how many messages each user has sent in the last minute and day. Over the limit, the server refuses with `429 Too Many Requests` and a `Retry-After` header saying how long to wait. That is **rate limiting**, and it stops one user from running up a large AI bill.

8. Authentication with JWT

When you sign in, the server needs a way to recognise you on every later request without asking for your password again. This project uses **JSON Web Tokens (JWT)**. Think of a festival wristband: it proves you paid, staff can check it at a glance, and it expires.

You get two tokens:

- an **access token**, which lasts 15 minutes. It's sent with every request in a header: `Authorization: Bearer <token>`.
- a **refresh token**, which lasts 7 days. Its only job is to get a fresh access token when the old one expires. Each use gives you a *new* refresh token and cancels the old one ("rotation").

The frontend handles all of this for you. If a request fails with `401`, it quietly refreshes the token once and tries again. Signing out *blacklists* the refresh token on the server, so it can never be used again.

9. Environment variables and secrets

Settings that differ between computers or must stay secret (passwords, API keys) live outside the code in **environment variables**. In development they sit in a `.env` file, which Git ignores so secrets never get committed. You copy the safe template `.env.example` and fill it in.

10. Sync vs async, WSGI vs ASGI

A **synchronous** server is like a phone support line: each worker handles one caller at a time. An **asynchronous** server is like live chat: one agent juggles many conversations. A streaming reply holds its connection open for tens of seconds, which would tie up a synchronous (**WSGI**) worker the whole time. So the chat endpoint is *async* and runs on an **ASGI** server, **uvicorn**. That's why the project uses `make run` (uvicorn) rather than Django's usual `runserver`.

PART I · CHAPTER 4

The Technology Stack

Here is every major tool the project uses, what it's for, and where you'll find it. You don't need to learn them all at once; come back to this chapter whenever you meet an unfamiliar name.

Frontend (runs in the browser)

Tool	What it does here	Where
React 19	Builds the user interface from small reusable pieces called <i>components</i> .	<code>frontend/src/</code>
Vite 8	Development server with instant reload; bundles the app for production.	<code>vite.config.js</code>
Tailwind CSS 4	Styling with utility classes such as <code>rounded-xl</code> and <code>px-4</code> .	<code>src/index.css</code>
React Router 7	Maps addresses like <code>/login</code> and <code>/c/:id</code> to pages.	<code>src/App.jsx</code>
react-markdown + remark-gfm	Turns the model's Markdown into safe HTML, including tables.	<code>components/Markdown.jsx</code>
react-syntax-highlighter	Colours code blocks (Prism, 15 languages).	<code>components/CodeBlock.jsx</code>
Vitest + Testing Library	Automated frontend tests.	<code>*.test.jsx</code>
eslint	Catches common JavaScript mistakes.	<code>.eslinttrc.json</code>

Backend (runs on the server)

Tool	What it does here	Where
Python 3.12+	The language. 3.13 is recommended; CI tests on 3.12.	
Django 6	Web framework: URLs, models, the admin site, security middleware.	<code>backend/config/</code>
Django REST Framework	Builds the JSON API: serializers, viewsets, pagination, throttling.	<code>*/views.py</code> , <code>*/serializers.py</code>
SimpleJWT	Issues, refreshes and blacklists JWT tokens.	<code>users/</code>
google-gemai	Google's official Python SDK for Gemini.	<code>chat/services/gemini.py</code>
uvicorn / gunicorn	ASGI servers. Uvicorn alone in development; gunicorn managing uvicorn workers in production.	<code>Makefile</code> , <code>deploy/</code>
psycopg 3	Connects Python to PostgreSQL.	
django-redis	Uses Redis as Django's cache (for rate-limit counters).	<code>settings/base.py</code>
django-envirom	Reads settings from <code>.env</code> .	<code>settings/base.py</code>
drf-spectacular	Generates the API documentation page at <code>/api/docs/</code> .	<code>openapi.yaml</code>

Tool	What it does here	Where
django-cors-headers	Controls which websites may call the API.	<code>settings/base.py</code>
argon2-cffi	Password hashing with Argon2.	
sentry-sdk	Optional error reporting in production.	<code>settings/prod.py</code>
pytest, ruff	Tests and linting/formatting.	<code>backend/tests/</code> , <code>pyproject.toml</code>

Data and AI services

PostgreSQL 16	The main database.
Redis 7	Cache for rate-limit counters.
Gemini models	<code>gemini-3.6-flash</code> is the main model. If it keeps failing, the server falls back to <code>gemini-3.5-flash-lite</code> , which is also used to write conversation titles.

Hosting and automation

GitHub Actions	Runs every test on every push; deploys the backend when <code>main</code> passes (<code>.github/workflows/ci.yml</code>).
Vercel	Builds and hosts the frontend.
A Linux VPS	Runs the backend behind nginx , managed by systemd (<code>deploy/</code>).
Docker Compose	Optional one-command local setup (<code>docker-compose.yml</code>).

PART II

Get It Running

Install the tools, start both halves of the app on your own computer, and learn your way around the folders.

5 Preparing Your Computer

6 Running the Project Locally

7 A Map of the Codebase

PART II · CHAPTER 5

Preparing Your Computer

You need six things installed and one API key. This chapter shows how to get each and how to check it worked. It takes about 20 minutes the first time.

Checklist

Tool	Version	Why you need it	Check with
Git	any recent	Download the code	<code>git --version</code>
Python	3.12 or newer (3.13 recommended)	Runs the backend	<code>python3 --version</code>
Node.js (with npm)	20 or newer (CI uses 22)	Runs the frontend tools	<code>node --version</code>
PostgreSQL	16	The database	<code>pg_isready</code>
Redis	7 or newer	Rate-limit counters	<code>redis-cli ping</code> → PONG
A code editor	any	VS Code is a good free choice	
Gemini API key		Lets the server call Gemini	see below

PREFER DOCKER?

If you have Docker Desktop, you can skip installing PostgreSQL, Redis and Python. Docker Compose runs all three for you (Chapter 6, "The Docker shortcut"). You still need Node.js for the frontend.

Installing on macOS

The easiest route is [Homebrew](#), the macOS package manager. Once Homebrew is installed:

Terminal · macOS

```
# Install everything in one go
brew install git python@3.13 node postgresql@16 redis

# Start the database and Redis now, and every time the Mac boots
brew services start postgresql@16
brew services start redis

# Check they are running
pg_isready          # → accepting connections
redis-cli ping      # → PONG
```

Installing on Ubuntu / Debian Linux

Terminal · Ubuntu

```
sudo apt update
sudo apt install git python3 python3-venv python3-pip postgresql redis-server
# Node from your distro is often too old; use NodeSource or nvm for Node 20+
sudo systemctl enable --now postgresql redis-server
```

Check the versions the package manager gives you. If Python or PostgreSQL is older than the checklist asks for, install a newer build (for example via the deadsnakes PPA for Python, or the official PostgreSQL apt repository).

Windows

Use **WSL 2** (Windows Subsystem for Linux) and follow the Ubuntu steps inside it, or use Docker Desktop. The project's `Makefile` commands assume a Unix-style shell.

Getting a Gemini API key

- 1 Go to aistudio.google.com/apikey and sign in with a Google account.
- 2 Click **Create API key** and copy the key.
- 3 Keep it somewhere safe for now. In the next chapter you'll paste it into `backend/.env`.

TREAT THE KEY LIKE A PASSWORD

Anyone with your key can make requests that Google bills to your account. Never paste it into frontend code, a chat message, a screenshot or a Git commit. If it leaks, delete it in AI Studio and create a new one.

Where to put the project folder

MACOS: AVOID ICLOUD-SYNCED FOLDERS

If "Desktop & Documents Folders" syncing is turned on in iCloud, don't clone into `~/Desktop` or `~/Documents`. iCloud tries to sync the thousands of files in `node_modules` and `.venv`, and the tests stall or time out. Use a folder like `~/Projects` instead.

PART II · CHAPTER 6

Running the Project Locally

You'll start two programs side by side: the Django backend on port 8000 and the React frontend on port 5173. Follow the steps in order; each one ends with a way to check it worked.

Step 1 · Download the code

Terminal

```
mkdir -p ~/Projects && cd ~/Projects
git clone https://github.com/tausif1337/llm-powered-ai-chatbot.git
cd llm-powered-ai-chatbot
```

Step 2 · Create the database

The backend expects a PostgreSQL user called `chatbot` with password `chatbot`, and a database also called `chatbot`. That's what the default `DATABASE_URL` in `.env.example` points to. Open the PostgreSQL shell:

Terminal

```
psql postgres          # on Linux: sudo -u postgres psql
```

Then type these SQL commands at the `postgres=#` prompt:

psql

```
CREATE ROLE chatbot WITH LOGIN PASSWORD 'chatbot' CREATEDB;
CREATE DATABASE chatbot OWNER chatbot;
\q
```

WHY CREATEDB?

The test suite creates a temporary database called `test_chatbot` each time it runs, so the `chatbot` user needs permission to create databases. The project README only says `createdb chatbot`; that creates the database but not the `chatbot` user, so the steps above are the safer route for a fresh machine.

Step 3 · Install the backend

Terminal

```
cd backend
make install          # creates .venv/ and installs all Python packages
```

`make install` creates a **virtual environment** in `backend/.venv`, a private folder of Python packages just for this project, and installs everything listed in `backend/requirements/dev.txt`. Run `make help` at any time to see every available command.

Step 4 • Create your .env file

Terminal (still in backend/)

```
cp .env.example .env

# Generate a random secret key and copy the output
.venv/bin/python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Open `backend/.env` in your editor and change two lines:

backend/.env

```
SECRET_KEY=paste-the-random-string-here
GEMINI_API_KEY=paste-your-gemini-key-here
```

Leave everything else as it is for now. Chapter 12 explains every setting.

Step 5 • Create the tables

Terminal

```
make migrate
```

You should see a list of lines ending in `OK`, one per migration. The database now has all its tables.

Step 6 • Start the backend

Terminal

```
# Important: tell Django to use the development settings (see the box below)
export DJANGO_SETTINGS_MODULE=config.settings.dev

make run                # → Uvicorn running on http://127.0.0.1:8000
```

DON'T SKIP THE EXPORT LINE

`make run` starts uvicorn with `config/asgi.py`, which falls back to the **production** settings when `DJANGO_SETTINGS_MODULE` isn't set in your shell. The `.env` file is read too late to change that choice. Production settings switch off debug mode and redirect every request to HTTPS, so the frontend's calls fail with network errors and `curl` shows a `301`.

You can make the setting permanent by adding the `export` line to your shell profile (`~/.zshrc` or `~/.bashrc`), or put it on the same line: `DJANGO_SETTINGS_MODULE=config.settings.dev make run`.

Leave this terminal running. Open a second terminal and check the server:

Second terminal

```
curl http://127.0.0.1:8000/healthz/
{"status": "ok"}

curl http://127.0.0.1:8000/readyz/
{"status": "ok", "checks": {"database": "ok", "cache": "ok"}}
```

`/healthz/` means "the process is alive". `/readyz/` also checks that PostgreSQL and Redis can be reached. If `readyz` says `degraded`, look at which check says `error` and see Chapter 17.

PORT 8000 ALREADY IN USE?

Run `PORT=8010 make run` instead, and set `VITE_PROXY_TARGET=http://127.0.0.1:8010` in `frontend/.env` in the next step.

Step 7 · Start the frontend

In the second terminal, from the project root:

Second terminal

```
cd frontend
npm install          # first time only; downloads packages into node_modules/
cp .env.example .env # defaults are right for local development
npm run dev          # → Local: http://localhost:5173/
```

Step 8 · Try it

Open <http://localhost:5173> in your browser, click **Create one**, register an account and send a message. If the reply streams in, everything is working.

HOW THE TWO HALVES FIND EACH OTHER

The browser only talks to `localhost:5173`. The Vite dev server forwards (*proxies*) anything under `/api`, `/healthz` and `/readyz` to Django on port 8000. That's set up in `frontend/vite.config.js`. Because the browser sees a single address, there are no cross-origin (CORS) problems in development.

Your daily routine

Terminal 1 · backend

```
cd backend
export
DJANGO_SETTINGS_MODULE=config.settings.dev
make run
```

Terminal 2 · frontend

```
cd frontend
npm run dev
```

Both servers reload automatically when you save a file. Press `Ctrl` + `C` in a terminal to stop its server.

Optional extras

The demo account

To get the one-click demo panel from Figure 2.1 locally:

```
Terminal (in backend/)

.venv/bin/python manage.py seed_demo          # create or refresh the account
.venv/bin/python manage.py seed_demo --reset  # ...and delete its conversations
```

Then add these two lines to `frontend/.env` and restart `npm run dev`:

```
frontend/.env

VITE_DEMO_EMAIL=demo@devtechguru.cloud
VITE_DEMO_PASSWORD=TryTheChatbot!2026
```

The admin site

```
Terminal (in backend/)

make superuser          # asks for an email and password
```

Now visit <http://127.0.0.1:8000/admin/> to browse users, conversations and messages. Under uvicorn in development the admin pages may appear without styling, because uvicorn doesn't serve Django's static files. Everything still works.

Interactive API documentation

LLM Powered AI Chatbot API 1.0.0 OAS 3.0

</api/schema/>

Django + DRF backend for the Gemini-backed chatbot.

Authorize

auth

- POST </api/auth/login/>
- POST </api/auth/logout/>
- GET </api/auth/me/>
- POST </api/auth/refresh/>
- POST </api/auth/register/>
- POST </api/auth/verify/>

Figure 6.1 <http://127.0.0.1:8000/api/docs/> lists every endpoint. Click **Authorize**, paste an access token, and you can try requests from the page.

The Docker shortcut

If you'd rather not install PostgreSQL, Redis and Python yourself, Docker Compose can run all three. You still need `backend/.env` with at least `SECRET_KEY` and `GEMINI_API_KEY` set (Step 4).

Terminal (project root)

```
docker compose up --build
# starts: db (PostgreSQL 16), redis (Redis 7), backend (Django on :8000)
# the backend runs migrations automatically before starting
```

Then start the frontend exactly as in Step 7; the frontend is not part of the Compose file. Compose publishes PostgreSQL on port 5432 and Redis on 6379. If you already run them locally, stop the local copies first or the ports will clash.

PART II · CHAPTER 7

A Map of the Codebase

The repository has about 150 files, but you only need a handful to understand the app. This chapter is your map.

The top level

```
llm-powered-ai-chatbot/
```

backend/	Django API, Gemini service, streaming endpoint
frontend/	React single-page app
docs/	architecture, security, runbook, deployment, testing, ADRs, the PRD
deploy/	server files: systemd unit, nginx config, deploy script
.github/workflows/	ci.yml: tests on every push, backend deploy from main
docker-compose.yml	optional local PostgreSQL + Redis + backend
README.md	quick overview (partly out of date on hosting)

Backend

A Django project is made of a **project package** (settings and URLs) plus several **apps**, each a folder owning one area of the product.

```
backend/
```

config/	the project package
settings/base.py	settings shared by every environment
settings/dev.py	local development (debug on, browsable API)
settings/prod.py	production (HTTPS, HSTS, Sentry)
settings/test.py	tests (fast hashing, in-memory cache)
urls.py	the top-level URL table
asgi.py / wsgi.py	server entry points
common/	shared plumbing used by every app
exceptions.py	error codes + one JSON error shape
middleware.py	request IDs, access log
throttling.py	rate-limit classes
models.py	UUID + timestamp base classes
views.py	/healthz/ and /readyz/
users/	accounts: email login, register, JWT endpoints
management/commands/seed_demo.py	
conversations/	Conversation + Message models and their CRUD API
chat/	the streaming endpoint and usage accounting
views.py	POST /api/chat/ (the heart of the app)
models.py	UsageRecord
services/	
gemini.py	everything that talks to Google
context.py	chooses which history to send
prompts.py	system instruction + title prompt
streaming.py	formats SSE frames
persistence.py	async-safe database helpers
tests/	pytest suite

Makefile	shortcuts: install, run, test, lint...
openapi.yaml	the published API contract

Frontend

frontend/src/

main.jsx	entry point: applies theme, renders <App/>
App.jsx	routes: /login, /register, /, /c/:conversationId
index.css	Tailwind setup + colour tokens for light/dark
api/	
client.js	fetch wrapper: tokens, auto-refresh, ApiError
auth.js	register, login, logout, me
conversations.js	list, get, create, rename, delete
chat.js	streams /api/chat/ and parses SSE frames
auth/AuthContext.jsx	who is signed in, shared with every component
hooks/	
useChat.js	the chat state machine (idle → sending → streaming...)
useConversations.js	sidebar list with optimistic rename/delete
pages/	Login.jsx, Register.jsx, Chat.jsx
components/	ChatWindow, Message, ChatInput, Sidebar, Markdown, CodeBlock, ErrorBanner, ProtectedRoute, ...
theme/useTheme.js	system / light / dark switch
test/setup.js	test environment shims

"Where do I look when..."

You want to change...	Start in
How the assistant behaves or talks	backend/chat/services/prompts.py
Which Gemini model is used, temperature, retries	backend/.env , then chat/services/gemini.py
How much history is sent to the model	chat/services/context.py and <code>CONTEXT_MAX_*</code> in .env
The streaming endpoint's steps	backend/chat/views.py
Conversation or message fields	backend/conversations/models.py + serializers
Error codes and messages	backend/common/exceptions.py
Rate limits	<code>RATE_LIMIT_*</code> in .env ; common/throttling.py
The chat screen layout	frontend/src/pages/Chat.jsx , components/ChatWindow.jsx
How one message bubble looks	components/Message.jsx , Markdown.jsx , CodeBlock.jsx
What happens while a reply streams	frontend/src/hooks/useChat.js
Colours and theme	frontend/src/index.css
Deployment	deploy/ , .github/workflows/ci.yml , docs/DEPLOYMENT.md

READ THE COMMENTS

This codebase explains *why* in its comments, especially at the top of each file. When something looks odd, the reason is usually written a few lines above it.

PART III

How It Works

Follow a single message through the whole system, then look closely at the backend, the frontend and the database.

8 The Journey of One Message

9 Inside the Backend

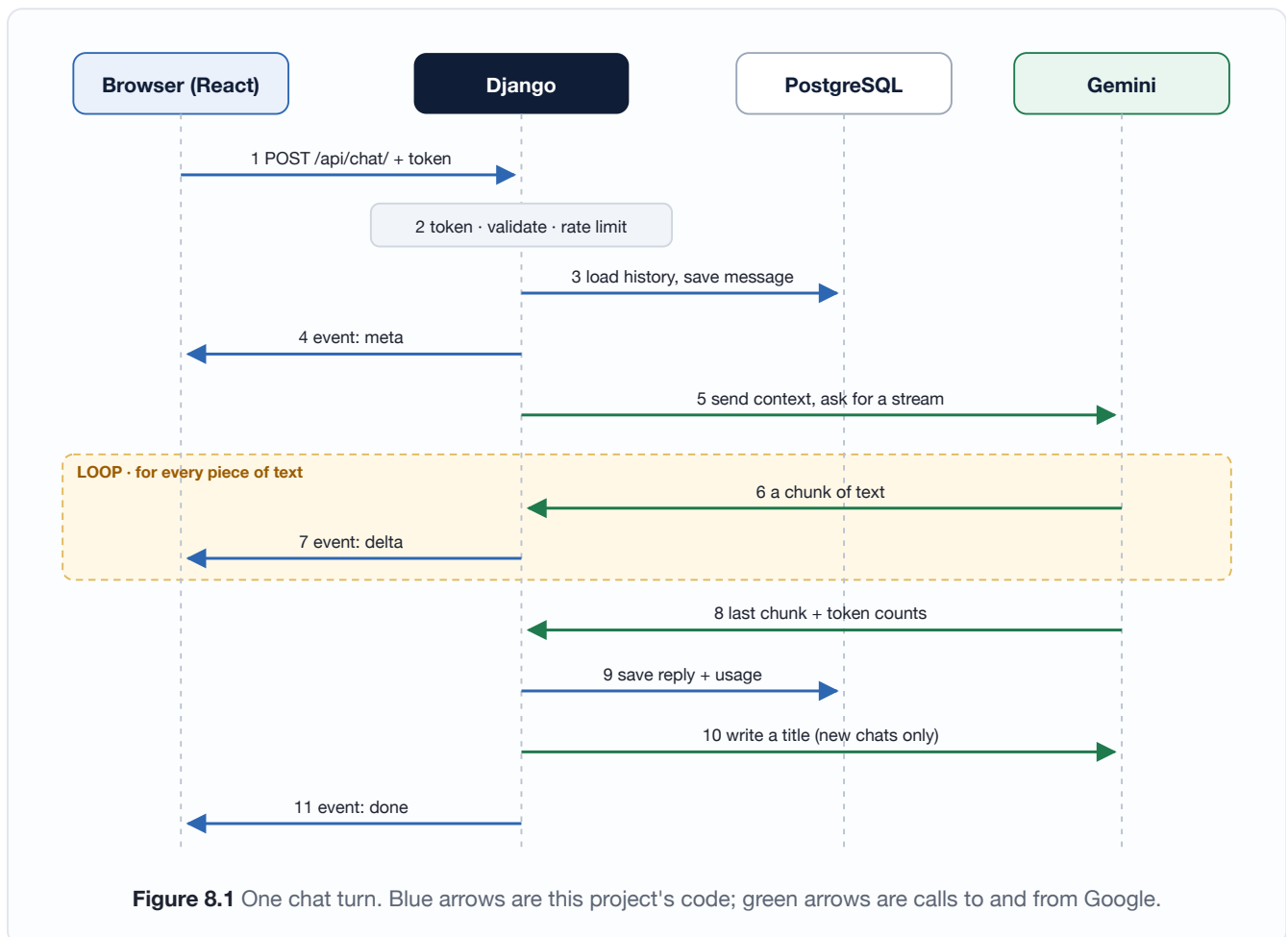
10 Inside the Frontend

11 The Data Model

PART III · CHAPTER 8

The Journey of One Message

You type "Explain Python decorators" and press Enter. About three seconds later the answer starts appearing. Here is everything that happens in between, with the file responsible for each step.



Step by step

In the browser

- 1 You press Enter.** `ChatInput.jsx` trims the text and calls `send()` from the `useChat` hook.
- 2 Instant feedback.** `useChat` adds your message and an empty assistant bubble to the screen straight away, then sets the status to `sending`, which shows the "Thinking" dots.
- 3 The request.** `api/chat.js` sends `POST /api/chat/` with `{"conversation_id": ..., "message": ...}` and the header `Authorization: Bearer <access token>`. It uses `fetch` rather than the browser's built-in `EventSource`, because `EventSource` can't send a POST body or an Authorization header.

On the server, before anything is sent back

- 4 **A request ID is attached.** `common/middleware.py` gives the request a unique ID (or reuses one sent in an `X-Request-ID` header). It appears in every log line and in any error you see.
- 5 **Who are you?** `chat/views.py` checks the JWT. If it's missing or invalid: `401`.
- 6 **Is the request sensible?** The body must be valid JSON under 256 KB; `message` must not be blank and must be at most 32,000 characters. Otherwise: `400` or `413`.
- 7 **Rate limits.** At most 20 messages a minute and 500 a day per user, counted in Redis. Over the limit: `429`.
- 8 **Which conversation?** If a `conversation_id` was sent, the server looks it up *among your conversations only*; someone else's ID gets the same `404` as a missing one. If none was sent, it creates a new conversation.
- 9 **Build the context.** `services/context.py` takes the system instruction, as much recent history as fits the token budget, and your new message (Chapter 9 shows how).
- 10 **Save your message** to the database.

The stream

- 11 **Headers go out.** Django starts a *streaming response* with status `200` and type `text/event-stream`. From this moment the status code can't change; later problems have to be reported inside the stream.
- 12 **`event: meta`** is sent first: the conversation ID (so a new chat's address can become `/c/<id>`), the model name, the request ID, and how many history messages were included or dropped.
- 13 **Gemini is called.** `services/gemini.py` asks Gemini for a streamed answer. If Google is briefly unavailable, it waits and retries (Chapter 9).
- 14 **`event: delta`** frames flow as Gemini produces text. Gemini's private "thinking" parts are filtered out; only answer text is forwarded. If Gemini goes quiet for 15 seconds, a `: heartbeat` keeps the connection alive.
- 15 **Save the reply.** When Gemini finishes, the full answer is saved as a `Message`, and a `UsageRecord` stores the token counts, timing and outcome.
- 16 **Title.** For a brand-new conversation, the server asks the lighter model for a 3 to 6 word title.
- 17 **`event: done`** carries the saved message's ID, the title and the token usage. The stream closes.

Back in the browser

- 18 `api/chat.js` reads the incoming bytes, splits them into frames at each blank line, and calls `onMeta`, `onDelta`, `onDone` or `onError`.
- 19 `useChat` appends each `delta` to the assistant bubble; React re-renders it through `Markdown.jsx`, so formatting appears as the text grows.
- 20 On `done`, the status becomes `completed`, the Send button comes back, and the sidebar title updates.

When things go wrong

The server **always** ends a stream with exactly one final frame, `done` or `error`. That promise is what lets the browser know for certain when a reply is over.

Situation	What the server does	What you see
Problem <i>before</i> streaming starts (bad token, empty message, rate limit, unknown conversation)	Normal JSON error with a real status code (<code>401</code> , <code>400</code> , <code>429</code> , <code>404</code> ...)	The empty bubble disappears; a red banner explains.
Gemini fails <i>before</i> the first word	Retries with backoff, then tries the fallback model	A slightly longer wait. If everything fails: an <code>error</code> frame.
Gemini fails <i>mid-answer</i>	No retry (you've already seen text). Saves the partial answer, sends an <code>error</code> frame.	The partial text stays, marked "incomplete", with the error below it.
Safety filter blocks the answer	Records outcome <code>blocked</code> , sends <code>error</code> with code <code>content_blocked</code>	"The response was blocked by a safety filter. Try rephrasing your message."
You press Stop or close the tab	Notifies the disconnect, saves whatever text was produced, records outcome <code>cancelled</code>	"Stopped." under the partial answer.
The connection drops with no final frame	(nothing reaches you)	The browser raises <code>stream_interrupted</code> : "The connection closed before the reply finished."

PART III · CHAPTER 9

Inside the Backend

The backend is a Django project with four apps and one firm rule: views handle the web side, and everything that knows about Gemini lives in a separate service layer.

The layers

HTTP (views)	Validate input, check who's asking, shape the response. Never import Google's SDK. <code>*/views.py</code>
Service	All Gemini knowledge: config, retries, fallback, error translation. Returns the project's own objects and exceptions. <code>chat/services/</code>
Persistence	Database helpers the async chat view can call safely. <code>chat/services/persistence.py</code>
Models	Table definitions, plus the <code>for_user()</code> filter that keeps data private. <code>*/models.py</code>

Because the layers are separate, you could swap Gemini for another AI provider by rewriting `gemini.py`, without touching a single view.

Settings: one base, three variants

`config/settings/base.py` reads every value from the environment. Three small files build on it:

File	Used when	What it changes
<code>dev.py</code>	<code>manage.py</code> commands by default; <code>make run</code> once you export it	Debug on, browsable API, no HTTPS redirect
<code>prod.py</code>	Default for <code>asgi.py</code> / <code>wsgi.py</code> ; the Dockerfile	Debug off, HTTPS redirect, HSTS, secure cookies, Sentry
<code>test.py</code>	pytest (set in <code>pyproject.toml</code>)	Fast password hashing, in-memory cache, no throttles

Every request's first stops: middleware

Middleware is code that wraps every request, like airport security before the gate. In order:

`RequestIDMiddleware` (tags the request), Django's security, CORS, session, CSRF, auth and clickjacking middleware, and finally `AccessLogMiddleware`, which writes one log line per request with method, path, status and duration. It never logs message content.

The users app

A custom `User` model logs in with **email** instead of a username, has a UUID primary key, and lowercases emails on save. Passwords are hashed with **Argon2**, a modern algorithm designed to be slow for attackers.

Endpoint	Does
POST /api/auth/register/	Create an account; returns <code>user</code> plus <code>access</code> and <code>refresh</code> tokens (201).
POST /api/auth/login/	Same response shape as register, so the frontend needs one round trip.
POST /api/auth/refresh/	Trade a refresh token for a new access and refresh token.
POST /api/auth/logout/	Blacklist the refresh token (205). Safe to call twice.
GET /api/auth/me/	Who am I? Used to check a stored token on page load.
POST /api/auth/verify/	Check that a token is valid.

The conversations app

`ConversationViewSet` in `conversations/views.py` gives the usual list, create, retrieve, update and delete operations, plus a paged `/messages/` view. Three details are worth knowing:

- **Privacy is built into the query.** Every read goes through `Conversation.objects.for_user(request.user)`. Another user's conversation is never in the result set, so there's no permission check to forget, and asking for it returns `404` rather than `403`. That way an attacker can't even learn that an ID exists.
- **Deleting is "soft".** `DELETE` sets a `deleted_at` timestamp instead of removing the row, so history can be recovered.
- **Cursor pagination.** Lists come back in pages of 25 conversations (or 50 messages) with `next` and `previous` links, which stay stable even while new rows are being added.

The chat endpoint

`chat/views.py` is the most important file in the backend. Its class `ChatStreamView` is a plain *async* Django view, not a DRF view like the rest, because DRF is synchronous and would tie up a worker for the whole length of a reply. The cost is that the view has to do authentication, validation and throttling itself, which you can see as a series of early `return` statements at the top of `post()`.

backend/chat/views.py (simplified)

```

async def post(self, request):
    user = await _authenticate(request)           # → 401
    payload = json.loads(request.body)           # → 400 / 413
    serializer.is_valid(raise_exception=True)     # → 400
    wait = await _check_throttles(request)        # → 429 + Retry-After
    conversation = await db.get_conversation(...) # → 404, or create a new one
    context = build_contents(history, message, system_instruction)
    await db.save_user_message(conversation, message)
    return StreamingHttpResponse(self._event_stream(...),
                                content_type="text/event-stream")

```

`_event_stream()` is an *async generator*: a function that `yield`s one SSE frame at a time. Django sends each frame to the browser as soon as it's yielded. The response also sets `X-Accel-Buffering: no` and `Cache-Control: no-cache, no-transform` so proxies such as nginx pass frames straight through instead of collecting them.

The Gemini service

`GeminiService` in `chat/services/gemini.py` wraps Google's SDK. Its settings come from `.env`:

Model / fallback	<code>gemini-3.6-flash</code> / <code>gemini-3.5-flash-lite</code>
Temperature	0.7
Max output tokens	8192
Thinking level	<code>low</code> (cheaper and faster; <code>high</code> reasons more)
Timeout	60 seconds
Safety filters	Hate speech, dangerous content, sexually explicit, harassment: blocked only at <i>high</i> probability

Retries, backoff and fallback

AI services have bad moments. The service classifies each failure:

- **Worth retrying:** timeouts and HTTP `429`, `500`, `502`, `503`, `504`. It tries up to 3 times per model (`GEMINI_MAX_RETRIES`), waiting a random time between 0 and 0.5 s, then 0 to 1 s, and so on, capped at 8 s. This is **exponential backoff with jitter**; the randomness stops many servers from retrying in lockstep.
- **Not worth retrying:** anything else, such as a `400` for a bad parameter. It fails immediately and logs Google's reason as `upstream_detail`.
- **Fallback:** if the main model uses up its retries, the whole sequence runs again on the fallback model. The usage record notes `used_fallback`.
- **Mid-stream failures are never retried**, because restarting would make the text on your screen jump or repeat.

Google's error text is logged but never sent to the browser. The service turns it into one of the project's own exceptions (`GeminiError`, `GeminiTimeoutError`, `GeminiRateLimitError`), each with a stable code and a safe message.

A LESSON THE PROJECT LEARNED THE HARD WAY

An early version switched thinking off with `thinking_budget = 0`. Every request failed with `HTTP 400`: Gemini 3.x models reject that parameter even though the SDK accepts it. The fix was `thinking_level`, plus *integration tests* that call the real API (`make integration`). Re-run them whenever you change the model or its settings. See [docs/adr/0004-thinking-level-not-budget.md](#).

Building the context

`chat/services/context.py` decides what the model gets to read. It can't send unlimited history: it would cost more with every message and eventually overflow the model's context window. The algorithm:

1. Estimate tokens as `characters ÷ 4`. It's a rough guess, but it's only used for trimming.
2. Start with a budget of `CONTEXT_MAX_TOKENS` (32,000), minus the system instruction and your new message.
3. Walk the history **from newest to oldest**, keeping messages while they fit, up to `CONTEXT_MAX_MESSAGES` (40).

4. Put the kept messages back in time order and add your new message at the end.



The oldest turns are dropped first; the newest always survive.

If your new message is so large that even on its own it doesn't fit, the server returns `413` with code `context_too_large`: "This conversation is too long to continue. Start a new conversation."

Prompts

`chat/services/prompts.py` keeps the assistant's instructions as plain data. The default tells the model to lead with the answer, use Markdown with language-tagged code blocks, prefer runnable examples, state its assumptions, admit what it doesn't know, and never invent APIs or citations. A conversation can override this through its `system_instruction` field. `TITLE_INSTRUCTION` asks for a 3 to 6 word plain-text title.

Errors: one shape, stable codes

`common/exceptions.py` makes every non-streaming error look the same:

Error response body

```
{"error": {"code": "rate_limited",
           "message": "Rate limit exceeded. Try again in 12 seconds.",
           "request_id": "3f9c0b..."}}
```

Code	Status	Meaning
<code>validation_error</code>	400	Bad input. <code>details</code> lists problems per field.
<code>authentication_error</code>	401	Not signed in, or the token is invalid or expired.
<code>not_found</code>	404	Doesn't exist, or isn't yours (same answer on purpose).
<code>context_too_large</code>	413	The message doesn't fit the context budget.
<code>rate_limited</code>	429	Our own limit. Check <code>Retry-After</code> .
<code>upstream_error</code>	502	Gemini failed.
<code>upstream_timeout</code>	504	Gemini was too slow.
<code>upstream_rate_limited</code>	429	Google is throttling the project.
<code>database_error</code>	503	Storage problem.
<code>internal_error</code>	500	Unexpected bug; details are in the server log.
<code>permission_denied</code>	403	Reserved. Ownership failures are reported as <code>404</code> instead.

Two more codes only appear in the stream: `content_blocked` (safety filter) and, on the browser side, `stream_interrupted` and `network_error`.

Rate limits

Scope	Default	Applies to
<code>chat_minute</code>	20 / minute	Messages per user (burst protection)
<code>chat_day</code>	500 / day	Messages per user; the real cost ceiling

Scope	Default	Applies to
auth	10 / minute	Signed-in users on endpoints without their own scope
anon	5 / minute	Anonymous requests (register, login) per IP address
conversations	120 / minute	Sidebar operations per user

Health checks and logs

`/healthz/` answers "is the process up?" and checks nothing else. `/readyz/` runs `SELECT 1` against PostgreSQL and a write-and-read against Redis, and returns `503` if either fails. Load balancers and the deploy script use it.

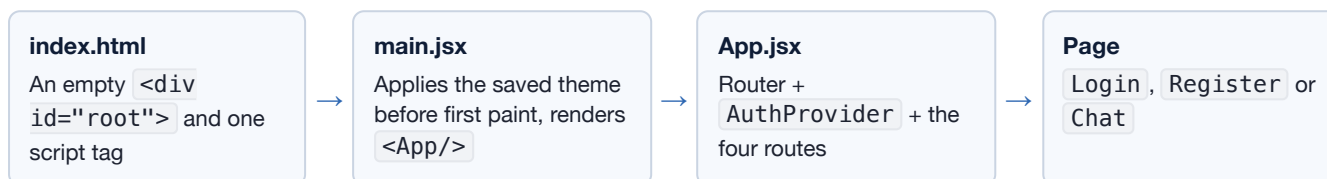
Logs are plain text locally and one JSON object per line elsewhere (whenever `ENVIRONMENT` isn't `local`). Every line carries the `request_id`, so one ID from a user's error banner finds every related log line. Useful event names include `gemini_retry`, `gemini_request_rejected`, `gemini_model_exhausted`, `gemini_stream_broken_midstream`, `chat_stream_cancelled` and `context_trimmed`.

PART III · CHAPTER 10

Inside the Frontend

The frontend is a single-page React app: one HTML page whose content JavaScript swaps as you move around. Its hardest job is reading the streamed reply and keeping the screen in step with it.

How the app starts



Address	Shows
<code>/login</code> , <code>/register</code>	The auth pages. A signed-in visitor is sent on to <code>/</code> .
<code>/</code>	Chat page with a new, empty conversation (sign-in required)
<code>/c/:conversationId</code>	Chat page with that conversation loaded (sign-in required)
anything else	Redirects to <code>/</code>

Knowing who's signed in

`auth/AuthContext.jsx` uses React *context*, a way to share values with every component without passing them down by hand. It provides `user`, `isAuthenticated`, `login`, `register`, `logout` and `bootstrapping`.

On page load, if a token is stored, the provider calls `GET /api/auth/me/` to confirm it still works. Meanwhile `bootstrapping` is `true` and `ProtectedRoute.jsx` shows "Loading..." instead of redirecting. Without that pause, a signed-in user would flash to the login screen on every reload.

Talking to the API

`api/client.js` handles two things the rest of the app shouldn't have to think about:

1. **Tokens.** They're stored in `localStorage` under `chatbot.access`, `chatbot.refresh` and `chatbot.user`. Every storage call is wrapped in `try/catch` because private browsing can block storage; the app still works, it just forgets you on reload.
2. **Refresh on 401.** When a request comes back `401`, `request()` calls `refreshAccessToken()` once and retries. If several requests fail together, they all share a single refresh call. If the refresh fails too, the tokens are cleared and you're signed out.

Any error body is turned into an `ApiError` with `code`, `message`, `status`, `requestId` and `fieldErrors`, which is exactly what `ErrorBanner` displays.

Reading the stream

`api/chat.js` sends the POST with `fetch`, then reads `response.body` chunk by chunk. Network chunks don't line up with SSE frames: one chunk can hold half a frame or three frames. So the code keeps a buffer:

```
frontend/src/api/chat.js (simplified)

buffer += decoder.decode(value, { stream: true })
const { frames, rest } = splitFrames(buffer) // split at each blank line
buffer = rest                                // keep the unfinished part
for (const raw of frames) {
  const parsed = parseFrame(raw)              // { event, data } or null for heartbeats
  switch (parsed.event) { case 'meta': ... case 'delta': ... case 'done': ... case 'error': ... }
}
```

If the stream ends without a `done` or `error` frame, the client reports `stream_interrupted` rather than treating a cut-off answer as complete.

The chat state machine

`hooks/useChat.js` tracks the chat as a small **state machine**: it's always in exactly one named state, and only certain events move it to another.

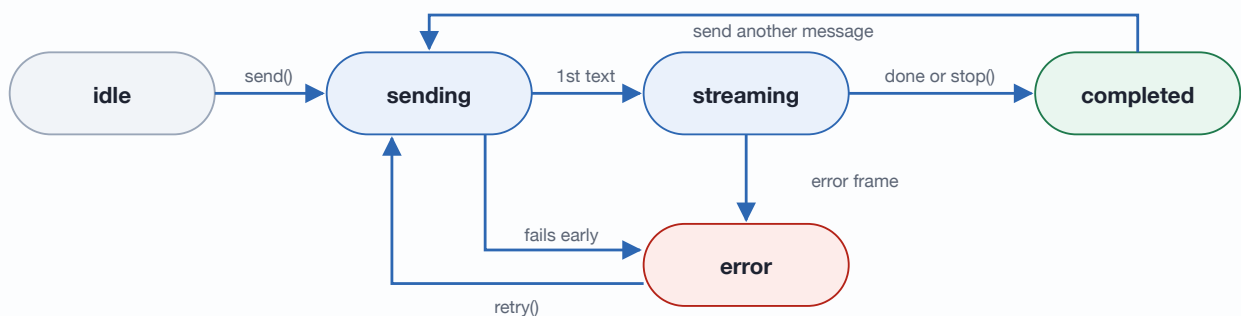


Figure 10.1 `useChat` states. "sending" covers the wait before the first word, which is when the Thinking dots show.

The hook returns `messages`, `status`, `error`, `send()`, `stop()`, `retry()` and `isBusy`. A few subtleties are handled carefully, and the comments in the file explain each one:

- **Stop** uses an `AbortController` to cancel the `fetch`; the server notices and saves the partial answer.
- **Retry** removes the failed exchange from the screen, then sends your last message again.
- **New conversations.** When `meta` brings a new conversation ID, `Chat.jsx` changes the address to `/c/<id>` without reloading the history. Reloading mid-stream would wipe the reply that's still arriving.

The sidebar list

`hooks/useConversations.js` loads the list and makes rename and delete **optimistic**: the screen changes immediately, and if the server then refuses, the change is rolled back and the list is reloaded. The app feels instant, and it stays correct if something fails.

Components at a glance

Component	Responsibility
<code>Chat</code> (page)	Glues sidebar, transcript and input together; loads history for <code>/c/:id</code>
<code>Sidebar</code>	Logo, New conversation, the list, your name, Sign out, theme toggle; slide-over on mobile
<code>ConversationList</code>	Date groups, rename-in-place, two-click delete
<code>ChatWindow</code>	Scrolling transcript, empty state with suggestions, "Jump to latest"
<code>Message</code>	One bubble: yours as plain text, the assistant's as Markdown; Copy and token count
<code>Markdown</code> / <code>CodeBlock</code>	Safe Markdown rendering; highlighted code with Copy
<code>ChatInput</code>	Auto-growing textarea, Enter to send, Send/Stop button, character counter
<code>LoadingIndicator</code>	The "Thinking" dots
<code>ErrorBanner</code>	Every user-visible error: message, Ref ID, Retry, Dismiss
<code>ProtectedRoute</code>	Sends signed-out visitors to <code>/login</code>
<code>DemoCredentials</code>	Demo panel, shown only when <code>VITE_DEMO_*</code> are set

Rendering model output safely

The model's text is untrusted: it could contain HTML such as `<script>`, whether by accident or because a user tricked it. `Markdown.jsx` uses `react-markdown`, which **escapes** HTML, so it appears as text instead of running. Links open in a new tab with `rel="noopener noreferrer nofollow"`. Your own messages are shown as plain text, never as Markdown. Chapter 16 lists the rules that keep it this way.

Styling and themes

Styles are Tailwind utility classes written directly in the JSX. Colours come from named *tokens* defined in `src/index.css`: `canvas`, `surface`, `raised`, `ink`, `ink-muted`, `line`, `accent`, `danger`, and `code-*` for code blocks. That's why you see classes like `bg-canvas` and `text-ink`. Dark mode redefines the same tokens, either when your system prefers dark or when the `data-theme="dark"` attribute is set on the page by the theme toggle (`theme/useTheme.js`, stored under `chatbot.theme`).

Accessibility built in

- The transcript is a `role="log"` region that screen readers announce politely as it updates; errors use `role="alert"`.
- Every button has an accessible name and a visible focus ring for keyboard users.
- Animations switch off for people who set "reduce motion" in their operating system.

PART III · CHAPTER 11

The Data Model

Four tables hold everything the app remembers. Understanding them makes the API, the admin site and the usage reports easy to read.

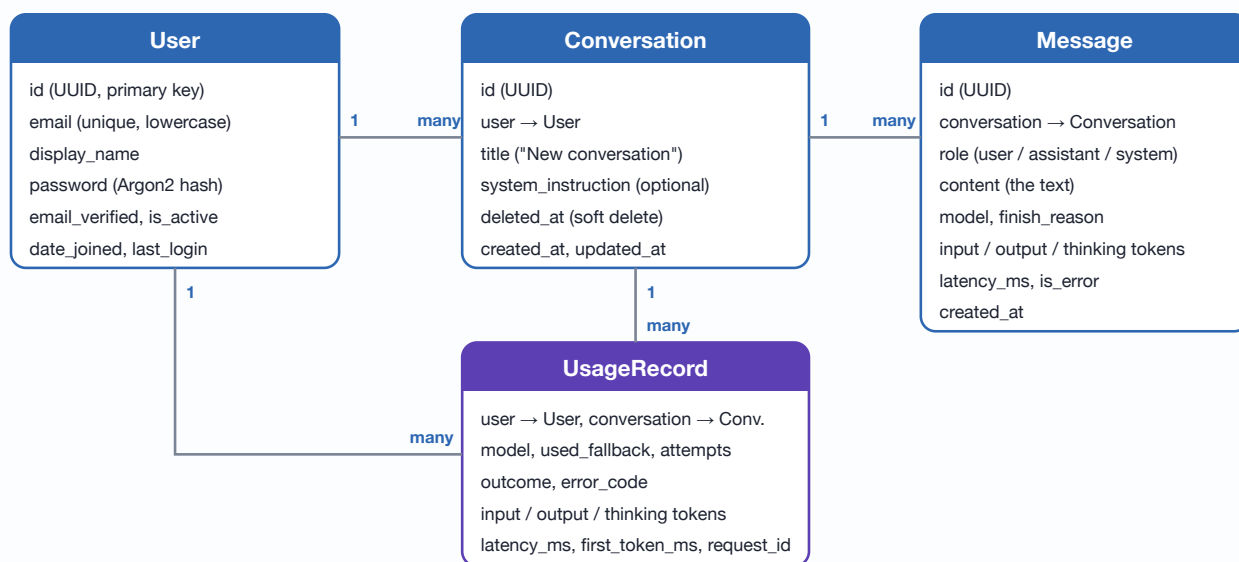


Figure 11.1 One user has many conversations; one conversation has many messages. Usage records point at both.

Things worth noticing

UUIDs, not 1, 2, 3

IDs look like `6b37a2a1-59e8-...`. Sequential numbers would let anyone guess other IDs and count how many chats exist. The PRD's example used an integer; this is a deliberate change (ADR 0003).

Soft delete

Deleting a conversation only fills in `deleted_at`. The `alive()` and `for_user()` filters hide it everywhere, but its messages are still there to recover.

Why a separate UsageRecord?

Failed, blocked and cancelled requests still cost money but produce no message. A separate table records *every* attempt, with outcome `success`, `error`, `blocked` or `cancelled`.

Indexes

Conversations are indexed by `(user, -updated_at)` for the sidebar; messages by `(conversation, created_at)` for history. Queries stay fast as tables grow.

Table names

Model	Database table
User	users_user
Conversation	conversations_conversation
Message	conversations_message
UsageRecord	chat_usage_record
(SimpleJWT)	token_blacklist_outstandingtoken , token_blacklist_blacklistedtoken

Looking at the data yourself

Three ways, from friendliest to most powerful:

1. **Admin site** at `/admin/` (after `make superuser`).
2. **Django shell**: `make shell`, then Python:

```
make shell

from conversations.models import Conversation
Conversation.objects.alive().count()
Conversation.objects.order_by("-updated_at").first().messages.all()
```

3. **SQL**: `psql postgres://chatbot:chatbot@localhost:5432/chatbot`, then for example:

```
psql · tokens used per model per day

SELECT model, date_trunc('day', created_at) AS day, count(*) AS requests,
       sum(input_tokens) AS input, sum(output_tokens) AS output,
       sum(thinking_tokens) AS thinking
FROM chat_usage_record GROUP BY 1, 2 ORDER BY 2 DESC;
```

CHANGING A MODEL

After editing a `models.py`, run `make migrations` to generate a migration file, read it, then `make migrate` to apply it. Commit the migration file along with your model change. CI fails if a model change has no migration.

PART IV

Build, Test, Ship

Configure the app, prove your changes work, make some changes of your own, and put it on the internet safely.

12 Configuration Reference

13 Testing

14 Your First Changes

15 Deployment

16 Security Essentials

17 Troubleshooting

PART IV · CHAPTER 12

Configuration Reference

All backend settings come from environment variables, which locally means `backend/.env`. The frontend has its own small `frontend/.env`. Restart the server after changing either.

Backend: Django core

Variable	Default / example	Meaning
<code>DJANGO_SETTINGS_MODULE</code>	<code>config.settings.dev</code>	Which settings file to use. Export it in your shell for <code>make run</code> (Chapter 6).
<code>SECRET_KEY</code>	<i>required</i>	Signs JWTs and other secrets. Long, random, and different in every environment.
<code>DEBUG</code>	<code>True</code> locally	Detailed error pages. Always off in production (<code>prod.py</code> forces it).
<code>ALLOWED_HOSTS</code>	<code>localhost, 127.0.0.1</code>	Host names the server answers to. Required in production.
<code>ENVIRONMENT</code>	<code>local</code>	Anything other than <code>local</code> switches logs to JSON; also prefixes cache keys.
<code>LOG_LEVEL</code>	<code>DEBUG</code> / <code>INFO</code>	How chatty the logs are.

Backend: Gemini

Variable	Default	Meaning
<code>GEMINI_API_KEY</code>	<i>required</i>	Your key from AI Studio. Server side only.
<code>GEMINI_MODEL</code>	<code>gemini-3.6-flash</code>	Main model. <code>gemini-2.5-*</code> models are unavailable to new keys.
<code>GEMINI_FALLBACK_MODEL</code>	<code>gemini-3.5-flash-lite</code>	Used when the main model exhausts its retries, and for titles.
<code>GEMINI_TIMEOUT_SECONDS</code>	<code>60</code>	Per-request timeout.
<code>GEMINI_MAX_RETRIES</code>	<code>3</code>	Attempts per model before giving up or falling back.
<code>GEMINI_MAX_OUTPUT_TOKENS</code>	<code>8192</code>	Longest reply allowed.
<code>GEMINI_TEMPERATURE</code>	<code>0.7</code>	Randomness, 0 to 2.
<code>GEMINI_THINKING_LEVEL</code>	<code>low</code>	<code>low</code> or <code>high</code> ; empty uses the model's default. Don't use the old <code>GEMINI_THINKING_BUDGET</code> : Gemini 3.x rejects it.
<code>CONTEXT_MAX_TOKENS</code>	<code>32000</code>	History budget (estimated tokens).
<code>CONTEXT_MAX_MESSAGES</code>	<code>40</code>	Most history messages ever sent.

Backend: data, auth and limits

Variable	Default	Meaning
<code>DATABASE_URL</code>	local <code>chatbot</code> DB	Where PostgreSQL is: <code>postgres://user:password@host:port/database</code>
<code>DB_CONN_MAX_AGE</code>	<code>60</code>	Seconds to keep a database connection open for reuse.
<code>REDIS_URL</code>	local, db 0	Redis for the cache and rate limits. The number at the end is the Redis database.
<code>JWT_ACCESS_TOKEN_LIFETIME_MINUTES</code>	<code>15</code>	Access token life.
<code>JWT_REFRESH_TOKEN_LIFETIME_DAYS</code>	<code>7</code>	Refresh token life, i.e. how long you stay signed in while idle.
<code>JWT_ROTATE_REFRESH_TOKENS</code> , <code>JWT_BLACKLIST_AFTER_ROTATION</code>	<code>True</code>	Issue a new refresh token on each use and cancel the old one.
<code>CORS_ALLOWED_ORIGINS</code>	localhost:5173	Websites allowed to call the API from a browser. In production, the frontend's address.
<code>CSRF_TRUSTED_ORIGINS</code>	localhost:5173	Origins trusted for cookie-based forms (the admin site).
<code>RATE_LIMIT_CHAT_PER_MINUTE</code>	<code>20</code>	Messages per user per minute.
<code>RATE_LIMIT_CHAT_PER_DAY</code>	<code>500</code>	Messages per user per day.
<code>RATE_LIMIT_ANON_PER_MINUTE</code>	<code>5</code>	Anonymous requests per IP per minute.
<code>RATE_LIMIT_AUTH_PER_MINUTE</code>	<code>10</code>	The <code>auth</code> scope.
<code>STREAM_HEARTBEAT_SECONDS</code>	<code>15</code>	Silence before a <code>: heartbeat</code> is sent.
<code>SENTRY_DSN</code> , <code>SENTRY_TRACES_SAMPLE_RATE</code>	empty, <code>0.1</code>	Optional error reporting (production settings only).

Defined but not used yet: `.env.example` also lists `STREAM_TRANSPORT`, `STREAM_IDLE_TIMEOUT_SECONDS`, `CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND` and `FRONTEND_URL`. At this commit no code reads them (Celery isn't installed), so changing them has no effect. The `EMAIL_*` settings are wired into Django, but the app doesn't send any email yet.

Frontend

Variable	Default	Meaning
<code>VITE_API_BASE_URL</code>	empty	Leave empty in development (calls go through the proxy). In production, the backend's full address, e.g. <code>https://chatbot.devtechguru.cloud</code> .
<code>VITE_PROXY_TARGET</code>	<code>http://127.0.0.1:8000</code>	Where <code>npm run dev</code> forwards <code>/api</code> , <code>/healthz</code> and <code>/readyz</code> .
<code>VITE_DEMO_EMAIL</code> , <code>VITE_DEMO_PASSWORD</code>	unset	If both are set, the sign-in page shows the demo panel.

Every `VITE_` variable is public. Vite copies them into the JavaScript every visitor downloads, so never put a secret in one, least of all the Gemini key.

PART IV · CHAPTER 13

Testing

Automated tests are small programs that check the real program still behaves. Run them before every commit; GitHub runs them again on every push.

Running the tests

Backend (in backend/)

```
make test          # whole suite
make cover         # + coverage (75%)
make integration   # live Gemini API
make lint          # ruff checks
make fmt           # auto-fix, format
make check         # Django checks
```

Frontend (in frontend/)

```
npm test           # run once
npm run test:watch # on save
npm run cover      # + coverage (70%)
npm run lint       # oxlint
npm run build      # prod build
```

WHAT THE BACKEND TESTS NEED

The tests use PostgreSQL (the `chatbot` user must be able to create a `test_chatbot` database; see Chapter 6), but **not** Redis: the test settings use an in-memory cache, and in the main suite Gemini is replaced by a stand-in. The live contract tests in `test_integration_gemini.py` are the exception. They call the real API and spend a few tokens.

MAKE TEST CAN CALL THE REAL GEMINI API

The live tests carry an `integration` marker, but at this commit nothing excludes that marker from `make test`. They only skip themselves when no `GEMINI_API_KEY` is available, and Django copies `backend/.env` into the environment when it starts. So with a real key in `.env`, `make test` runs them too. To keep a run fully offline:

Terminal (in backend/)

```
.venv/bin/python -m pytest tests/ -q -m "not integration"
```

What the backend tests cover

File	Checks
<code>test_auth.py</code>	Register, login, logout and blacklisting, <code>me</code> , password rules
<code>test_conversations.py</code>	CRUD, one user can't see another's data, soft delete, pagination
<code>test_context.py</code>	History assembly, role mapping, trimming by tokens and by count
<code>test_gemini_service.py</code>	Retries, fallback, error translation, thinking-part filtering, safety blocks
<code>test_chat_stream.py</code>	The whole chat endpoint with Gemini stubbed: frames, errors, persistence
<code>test_seed_demo.py</code>	The demo-account command

File	Checks
<code>test_integration_gemini.py</code>	Live contract: both models accept our settings, streaming arrives in chunks, titles work

What the frontend tests cover

Test files sit next to the code they test. `chat.test.js` checks SSE frame splitting and parsing; `client.test.js` covers token refresh; `useChat.test.jsx` covers the state machine; `Markdown.test.jsx` proves that `<script>`, `<img onerror>` and `javascript:` links stay harmless; the page and component tests cover sign-in, the sidebar, the input and the error banner.

What CI runs



Writing your first backend test

`backend/tests/conftest.py` provides ready-made *fixtures*, pieces of test setup you request by naming them as function parameters: `user`, `other_user`, `auth_api` (an API client already signed in as `user`), `conversation` and `conversation_with_history`.

`backend/tests/test_conversations.py` (add at the end)

```
def test_rename_trims_whitespace(auth_api, conversation):
    response = auth_api.patch(
        f"/api/conversations/{conversation.pk}/",
        {"title": " Trip planning "},
        format="json",
    )
    assert response.status_code == 200
    conversation.refresh_from_db()
    assert conversation.title == "Trip planning"
```

Run just that test with `.venv/bin/python -m pytest tests/test_conversations.py -k trims -q`.

Writing your first frontend test

`frontend/src/components/ErrorBanner.test.jsx` (add a case)

```
import { render, screen } from '@testing-library/react'
import ErrorBanner from './ErrorBanner'

it('shows the request id so users can report it', () => {
  render(<ErrorBanner error={{ code: 'upstream_error', message: 'Oops', requestId: 'abc123' }} />)
  expect(screen.getByText('Ref abc123')).toBeInTheDocument()
})
```

GOOD HABIT

When you fix a bug, first write a test that fails because of the bug, then fix the code until the test passes. That test will stop the bug from coming back.

PART IV · CHAPTER 14

Your First Changes

Five small exercises, from easiest to hardest. Each one touches a different part of the system, and each ends with a way to check your work. Do them on a new Git branch: `git switch -c my-first-change`.

Exercise 1 · Change the starter suggestions Frontend · easy

1. Open `frontend/src/components/ChatWindow.jsx`.
2. Find the `SUGGESTIONS` list near the top and replace one entry with your own question.
3. Save. With `npm run dev` running, the browser updates on its own.

Check: click **New conversation**; your suggestion appears, and clicking it sends it. Then run `npm test` to confirm nothing else broke.

Exercise 2 · Give the assistant a personality Backend · easy

1. Open `backend/chat/services/prompts.py`.
2. Add a line to `DEFAULT_SYSTEM_INSTRUCTION`, for example `- End every answer with one short follow-up question.`
3. Bump `SYSTEM_PROMPT_VERSION` to `"1.1.0"`, so you can tell which prompt produced which answers later.
4. Unicorn reloads automatically; start a new conversation.

Check: the answers now end with a question.

Want a different personality for just one conversation? Send `PATCH /api/conversations/<id>/` with `{"system_instruction": "..."} (try it from /api/docs/)`. A per-conversation instruction replaces the default.

Exercise 3 · Highlight a new language Frontend · medium

Kotlin code currently shows as plain text. Add it:

```
frontend/src/components/CodeBlock.jsx
```

```
import kotlin from 'react-syntax-highlighter/dist/esm/languages/prism/kotlin' // 1. import

const LANGUAGES = {
  bash, css, diff, go, java, javascript, json, jsx, kotlin, markup, ... // 2. register
}

const ALIASES = {
  ...
  kt: 'kotlin', // 3. alias
}
```

Check: ask the bot for "a Kotlin data class example"; the block is now coloured. Run `npm test` and `npm run build`.

Exercise 4 • Show a rate limit in action Config · medium

1. In `backend/.env` set `RATE_LIMIT_CHAT_PER_MINUTE=3`.
2. Stop and restart `make run`. Settings are read at startup, and the auto-reloader doesn't watch `.env`.
3. Send four short messages within a minute.

Check: the fourth gets a red banner: *"Rate limit exceeded. Try again in N seconds."* In your browser's developer tools (Network tab), the response is `429` with a `Retry-After` header. Put the value back to `20` afterwards.

Exercise 5 • Add a field to the API Backend · harder

Messages store `thinking_tokens`, but the API doesn't return them. Expose them:

1. In `backend/conversations/serializers.py`, add `"thinking_tokens"` to `MessageSerializer.Meta.fields`. The field is already read-only because `read_only_fields = fields`.
2. Write a test in `tests/test_conversations.py` that creates an assistant message with `thinking_tokens=94` and asserts it appears in `GET /api/conversations/<id>/`.
3. Run `make test`, then `make schema` to update `openapi.yaml`, the published API contract.

Check: your test passes, and `git diff openapi.yaml` shows the new field.

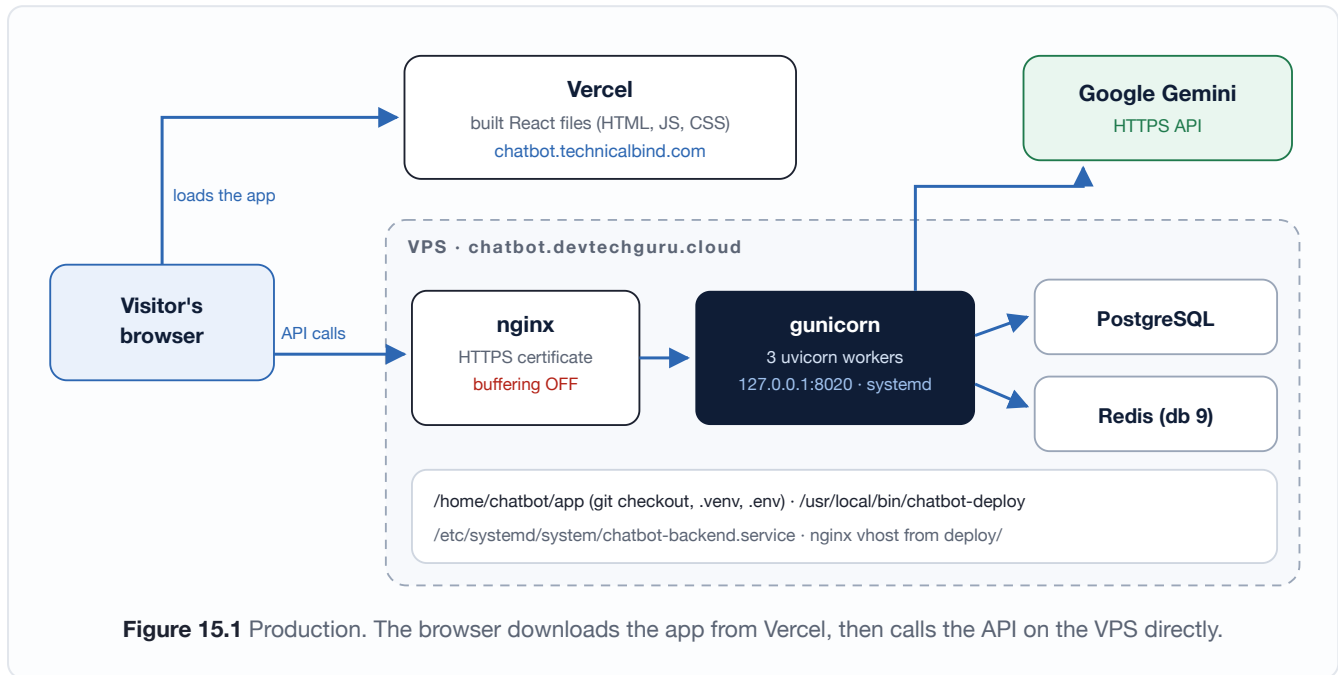
BEFORE YOU COMMIT

- ☐ `make lint` and `make test` pass (backend)
- ☐ `npm run lint` and `npm test` pass (frontend)
- ☐ No `.env` file or secret is included: check with `git status`
- ☐ If you changed a model, the new migration file is included

PART IV · CHAPTER 15

Deployment

In production, the two halves live in different places and deploy independently. The frontend is static files on Vercel; the backend is a long-running process on a Linux server.



Why not host everything on Vercel?

Vercel runs backends as short-lived *serverless functions*. The chat endpoint holds a connection open for a whole generation, needs Redis, and keeps a database connection pool. That calls for a long-running server process, so the backend lives on a VPS (a virtual private server, a Linux machine you rent).

The frontend on Vercel

- The Vercel project is linked to the GitHub repository with root directory `frontend/`. Every push to `main` triggers a fresh build (`npm run build` → `dist/`).
- `frontend/vercel.json` sends every unknown path to `index.html` (so `/c/<id>` works on refresh), caches built assets for a year, and adds security headers.
- Environment variables set in Vercel: `VITE_API_BASE_URL=https://chatbot.devtechguru.cloud`, plus the demo pair if the panel should show.

The backend on the VPS

The server is shared with other sites, so the backend follows its existing conventions rather than using Docker:

systemd `deploy/chatbot-backend.service` runs gunicorn with 3 uvicorn workers on `127.0.0.1:8020`, restarts it if it crashes, and limits which folders it can write to.

nginx	<code>deploy/nginx-chatbot.devtechguru.cloud.conf</code> handles HTTPS and forwards requests to port 8020. It also serves Django's static files for the admin.
PostgreSQL	The host's existing server, with a dedicated <code>chatbot</code> database and user.
Redis	The host's existing Redis, on database 9. Database 6 belongs to another app.

THE ONE NGINX SETTING THAT MATTERS MOST

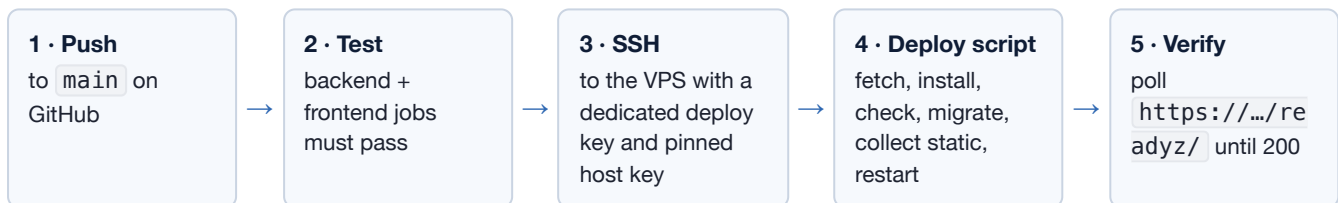
By default, a reverse proxy collects the response in large buffers before passing it on. For a streaming app, that means the whole answer arrives in one lump at the end. The project's nginx config switches this off:

```
deploy/nginx-chatbot.devtechguru.cloud.conf

proxy_buffering off;
proxy_request_buffering off;
proxy_set_header Connection ""; # SSE, not websockets
proxy_read_timeout 300;
```

Two gunicorn settings matter as well. `--timeout 120` has to stay above the Gemini timeout (60 s) plus retries, or gunicorn kills a worker mid-answer. And there's deliberately no `--max-requests`, because recycling a worker would cut off any reply it was streaming.

The CI/CD pipeline



The script is `deploy/deploy.sh`, installed on the server as `/usr/local/bin/chatbot-deploy`. It runs migrations *before* restarting, so new code never meets an old database schema. If the service doesn't become healthy within 30 seconds, it prints the last 40 log lines and fails the job. The deploy secrets (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`, `VPS_KNOWN_HOSTS`) live in the GitHub *production* environment.

Vercel doesn't go through GitHub Actions. It builds the frontend on its own whenever `main` changes.

Manual deploy and rollback

Terminal (maintainers with server access)

```
# Deploy the latest main
ssh root@<vps-host> /usr/local/bin/chatbot-deploy

# Roll back to an earlier commit
ssh root@<vps-host> /usr/local/bin/chatbot-deploy origin/<previous-commit-sha>
```

The real host is recorded in `docs/DEPLOYMENT.md`. Every migration so far only adds things, so rolling back code never requires touching the database.

Deploying your own copy

You don't need this exact server. The `backend/Dockerfile` builds a production image (gunicorn + uvicorn, honours `$PORT`, runs as a non-root user) that works on most container hosts. Whatever you choose, the backend needs:

- ☐ An ASGI server (gunicorn with uvicorn workers) and a proxy that doesn't buffer `text/event-stream`
- ☐ PostgreSQL and Redis reachable through `DATABASE_URL` and `REDIS_URL`
- ☐ `DJANGO_SETTINGS_MODULE=config.settings.prod`, a strong `SECRET_KEY`, and `ALLOWED_HOSTS` set to your API's host name
- ☐ `CORS_ALLOWED_ORIGINS` (and `CSRF_TRUSTED_ORIGINS`) set to your frontend's exact address, e.g. `https://chat.example.com`
- ☐ `GEMINI_API_KEY`, and `ENVIRONMENT=production` for JSON logs
- ☐ Migrations run on each deploy: `python manage.py migrate --noinput`
- ☐ HTTPS in front, since production settings redirect plain HTTP

And the frontend needs `VITE_API_BASE_URL` pointing at the backend, set *before* building, because Vite bakes it into the files at build time.

PART IV · CHAPTER 16

Security Essentials

A chatbot handles accounts, private conversations and a paid API key. Here's what protects them, what isn't done yet, and the rules you must keep when you change the code.

What's in place

Threat	Protection
Stolen Gemini key	The key lives only in server settings, is used only in <code>gemini.py</code> , and is never logged, serialised or sent to the browser.
Password theft	Argon2 hashing; at least 10 characters plus Django's password validators.
Reading someone else's chats	Every query is scoped with <code>for_user()</code> ; foreign IDs return <code>404</code> . UUIDs can't be guessed.
Session hijacking	15-minute access tokens; refresh tokens rotate and are blacklisted on use and on logout.
Malicious model output (XSS)	Markdown rendered with HTML escaped; no raw-HTML plugin; no <code>dangerouslySetInnerHTML</code> . CI fails the build if either appears.
Abuse and runaway cost	Per-user rate limits per minute and per day; 32,000-character message cap; 256 KB body cap.
Other websites calling the API	Explicit CORS allow-list, no wildcard.
SQL injection	The ORM builds all queries; the only raw SQL is the health check's <code>SELECT 1</code> .
Eavesdropping	HTTPS enforced, HSTS for one year, secure cookies (production settings).
Leaky errors	Users get a stable code and a safe message; Google's own error text stays in the server logs.
Private data in logs	Message content is never logged.

A known trade-off: tokens in localStorage

Tokens are kept in `localStorage` so you stay signed in after a reload. The cost: if an attacker ever managed to run JavaScript on the page (an XSS bug), they could read the tokens. The project limits the damage with short access tokens and rotation, and removes the most likely cause of XSS by never rendering untrusted HTML. The stronger alternative, an `httpOnly` refresh cookie, is written up in [docs/adr/0005-token-storage.md](#) for when it's needed.

Not implemented yet

These need a product decision before a wider launch: email verification and password reset (the fields exist, but no email is sent), multi-factor authentication, an audit log, redacting personal data from logs, prompt-injection defences beyond Gemini's safety settings, account deletion and data export, and a secrets manager.

Rules to keep when you change the code

NEVER

- ...put the Gemini key, or any secret, in frontend code or a `VITE_*` variable.
- ...commit a `.env` file. Only `.env.example` belongs in Git.
- ...add `rehype-raw` or use `dangerouslySetInnerHTML` to render model output.
- ...fetch a conversation with `Conversation.objects.get(pk=...)` in a view. Always go through `for_user(request.user)`.
- ...return Google's error text, or exception details, to the client.
- ...log message content.

ALWAYS

- ...validate input with a serializer on every new endpoint.
- ...add a test proving one user can't reach another user's new data.
- ...report vulnerabilities privately to the maintainers, not in a public GitHub issue.

PART IV · CHAPTER 17

Troubleshooting

Find your symptom, then work through the likely cause and fix. Two habits solve most problems: read the *whole* error message, and check `/readyz/`.

Setup and startup

Symptom	Likely cause	Fix
Frontend shows "Could not reach the server"; <code>curl</code> to the backend returns <code>301</code>	Backend started with production settings, which redirect to HTTPS	<code>export DJANGO_SETTINGS_MODULE=config.settings.dev</code> , then restart <code>make run</code> .
<code>ImproperlyConfigured: Set the SECRET_KEY environment variable</code>	No <code>backend/.env</code> , or it's missing <code>SECRET_KEY</code>	<code>cp .env.example .env</code> in <code>backend/</code> and fill it in.
<code>make: .venv/bin/python: No such file or directory</code>	Dependencies not installed yet	Run <code>make install</code> in <code>backend/</code> .
<code>connection refused</code> on port 5432	PostgreSQL isn't running	<code>brew services start postgresql@16</code> or <code>sudo systemctl start postgresql</code> .
<code>role "chatbot" does not exist</code> or <code>password authentication failed</code>	Database user not created	Chapter 6, Step 2. Or change <code>DATABASE_URL</code> to a user you have.
<code>/readyz/</code> shows <code>"cache": "error"</code> ; sign-in or chat fails with a server error	Redis isn't running. Rate limiting needs it on almost every request	<code>brew services start redis</code> ; check with <code>redis-cli ping</code> .
<code>address already in use</code>	Another program is using port 8000 or 5173	<code>PORT=8010 make run</code> and update <code>VITE_PROXY_TARGET</code> ; Vite will pick the next free port by itself.
Tests hang or time out on macOS	Project is in an iCloud-synced folder	Move it to e.g. <code>~/Projects</code> .
Tests fail: <code>permission denied to create database</code>	The <code>chatbot</code> user lacks <code>CREATEDB</code>	In psql: <code>ALTER ROLE chatbot CREATEDB;</code>

Chatting

Symptom	Likely cause	Fix
Every message fails with <code>upstream_error</code>	(1) A setting the model rejects, (2) a retired model name, or (3) a bad API key	Read <code>upstream_detail</code> in the backend log; it has Google's reason. Run <code>make integration</code> . Test the key with the <code>curl</code> command below.
The answer appears all at once instead of streaming	Something is buffering: <code>runserver</code> instead of <code>uvicorn</code> , or a proxy with buffering on	Use <code>make run</code> . In production, check the nginx settings in Chapter 15 and that nothing gzips <code>text/event-stream</code> .

Symptom	Likely cause	Fix
"Rate limit exceeded. Try again in N seconds."	Our own limit (20/min, 500/day)	Wait, or raise <code>RATE_LIMIT_CHAT_*</code> for local testing.
<code>upstream_rate_limited</code>	Google is throttling your API project	Check your quota in AI Studio; lower our per-minute limit so users get a clean 429 first.
"This conversation is too long to continue."	The new message alone exceeds the context budget	Send a shorter message, start a new conversation, or raise <code>CONTEXT_MAX_TOKENS</code> .
"The response was blocked by a safety filter."	Gemini's safety settings	Rephrase the question.
"The connection closed before the reply finished."	Network drop, or a proxy timeout shorter than the reply	Retry. For proxies, raise the read timeout above the heartbeat interval (15 s).
Signed out unexpectedly	Refresh token expired (7 days) or was blacklisted	Sign in again. That's expected behaviour.

Terminal · is my Gemini key valid?

```
curl -s -o /dev/null -w '%{http_code}\n' \
  -H "x-goog-api-key: $GEMINI_API_KEY" \
  https://generativelanguage.googleapis.com/v1beta/models
# 200 = the key works
```

Production

Symptom	Likely cause	Fix
Browser console: "blocked by CORS policy"	The frontend's address isn't in <code>CORS_ALLOWED_ORIGINS</code>	Add the exact origin (scheme + host, no trailing slash) and restart.
Frontend calls go to the wrong server	<code>VITE_API_BASE_URL</code> wrong, or changed without rebuilding	Fix it in Vercel and redeploy.
Refreshing <code>/c/<id></code> gives 404	Host isn't rewriting unknown paths to <code>index.html</code>	Keep the rewrite in <code>vercel.json</code> , or add one on your own host.
<code>readyz</code> reports <code>database: error</code>	Too many connections, or the database is down	Keep <i>workers</i> × <i>threads</i> below PostgreSQL's <code>max_connections</code> (see the runbook).
Health probe on the server returns 301	Production redirects plain HTTP	Probe with <code>-H 'X-Forwarded-Proto: https'</code> , as nginx does.

Finding the log line for an error

Every error banner shows a **Ref ID**. The same ID appears in the `X-Request-ID` response header and on every log line for that request:

Terminal

```
grep '"request_id":"3f9c0b..." app.log          # JSON logs (non-local)
journalctl -u chatbot-backend | grep 3f9c0b      # on the VPS
```

[docs/RUNBOOK.md](#) goes further: incident playbooks, cost investigations, and SQL for latency percentiles.

APPENDIX A

Glossary

Short definitions of the terms used in this guide.

Access token

A short-lived (15 min) JWT sent with each request to prove who you are.

ADR

Architecture Decision Record. A short document explaining one design choice and its trade-offs ([docs/adr/](#)).

API / endpoint

The set of addresses a program can call, and one such address, e.g. `/api/chat/`.

ASGI / WSGI

The async and sync standards for connecting Python web apps to servers.

Async generator

A Python function that `yield`s values over time; used to produce SSE frames.

Backoff (with jitter)

Waiting longer after each failed retry, with some randomness added.

Blacklist (token)

A server-side list of refresh tokens that may no longer be used.

CI/CD

Continuous integration and deployment: tests and deploys that run automatically on each push.

Component

A reusable piece of React UI, written as a function that returns JSX.

Context window

The most tokens a model can read at once.

CORS

Browser rules about which websites may call an API on another domain.

Cursor pagination

Paging with opaque "next/previous" links instead of page numbers; stable while data changes.

Environment variable

A named setting supplied from outside the code, e.g. from `.env`.

Fallback model

A second, cheaper model tried when the main one keeps failing.

Fixture

Reusable test setup that pytest passes into a test by name.

Frame (SSE)

One event in a stream: `event:` and `data:` lines followed by a blank line.

Heartbeat

A tiny comment frame sent during silence to keep the connection open.

Hook

A React function starting with `use` that holds state or side effects, e.g. `useChat`.

Idempotency key

An optional ID so a retried request isn't processed twice. `/api/chat/` accepts one, but at this commit the server doesn't use it.

JSX

HTML-like syntax inside JavaScript, used by React.

JWT

JSON Web Token: a signed token carrying who you are and when it expires.

LLM

Large language model, an AI that generates text token by token.

Middleware

Code that runs around every request in Django.

Migration

A generated file describing one change to the database structure.

Optimistic update

Changing the screen before the server confirms, and rolling back if it refuses.

ORM

Object-Relational Mapper: lets you use Python classes instead of SQL.

PRD

Product Requirements Document, the spec this project was built to.

Proxy (reverse proxy)

A server that forwards requests to another server; nginx in production, Vite in development.

Rate limiting / throttling

Capping how many requests a user can make in a time window.

Refresh token

A longer-lived (7 day) token used only to get new access tokens.

Request ID

A unique ID attached to each request, shown to users as "Ref", used to find logs.

Serializer

DRF class that validates incoming JSON and formats outgoing JSON.

Soft delete

Marking a row as deleted with a timestamp instead of removing it.

SPA

Single-page application: one HTML page whose content JavaScript updates.

SSE

Server-Sent Events, a simple standard for one-way streaming from server to browser.

State machine

A model where something is always in exactly one named state, like `sending`.

System instruction

Hidden guidance sent to the model with every request.

Temperature

How random the model's word choices are.

Thinking tokens

Tokens a model spends reasoning privately; billed but not shown.

Token (LLM)

A small piece of text, about four characters, that models read and write.

UUID

A 128-bit random identifier, practically impossible to guess.

Virtual environment

A private folder of Python packages for one project (`backend/.venv`).

VPS

Virtual private server: a rented Linux machine.

XSS

Cross-site scripting: an attack that runs someone else's JavaScript in your page.

APPENDIX B

Command Cheat Sheet

Everything you'll type regularly, on one page.

Backend • `cd backend`

<code>make install</code>	Create <code>.venv</code> , install packages
<code>make migrate</code>	Apply migrations
<code>make migrations</code>	Generate migrations after model edits
<code>make run</code>	Dev server on :8000 (export dev settings first)
<code>PORT=8010 make run</code>	...on another port
<code>make shell</code>	Django Python shell
<code>make superuser</code>	Create an admin user
<code>make test</code>	Run tests
<code>make cover</code>	Tests + coverage gate
<code>make integration</code>	Live Gemini contract tests
<code>make lint</code> / <code>make fmt</code>	Check / auto-fix style
<code>make check</code>	Django checks incl. deploy
<code>make schema</code>	Regenerate <code>openapi.yaml</code>
<code>make clean</code>	Delete caches and build files
<code>make help</code>	List all of the above

Demo account

<code>.venv/bin/python manage.py seed_demo</code>	Create / reset password
<code>... seed_demo --reset</code>	...and clear its chats

Frontend • `cd frontend`

<code>npm install</code>	Install packages
<code>npm run dev</code>	Dev server on :5173
<code>npm test</code>	Run tests once
<code>npm run test:watch</code>	Re-run tests on save
<code>npm run cover</code>	Tests + coverage gate
<code>npm run lint</code>	oxlint
<code>npm run build</code>	Production build into <code>dist/</code>
<code>npm run preview</code>	Serve the production build

Services

<code>docker compose up --build</code>	DB + Redis + backend
<code>docker compose down</code>	Stop them
<code>pg_isready</code>	Is PostgreSQL up?
<code>redis-cli ping</code>	Is Redis up? (PONG)
<code>curl :8000/healthz/</code>	Is Django up?
<code>curl :8000/readyz/</code>	Can it reach DB and Redis?

Useful addresses (local)

<code>localhost:5173</code>	The app
<code>127.0.0.1:8000/api/docs/</code>	API docs
<code>127.0.0.1:8000/admin/</code>	Admin site

APPENDIX C

API Quick Reference

All endpoints, plus a hands-on session with `curl`. For the full contract, see [backend/openapi.yaml](#) or [/api/docs/](#).

Method	Path	Auth	Purpose
POST	<code>/api/auth/register/</code>	no	Create account → <code>{user, access, refresh}</code> (201)
POST	<code>/api/auth/login/</code>	no	Sign in → <code>{access, refresh, user}</code>
POST	<code>/api/auth/refresh/</code>	no	<code>{refresh}</code> → new <code>{access, refresh}</code>
POST	<code>/api/auth/verify/</code>	no	Check a token
POST	<code>/api/auth/logout/</code>	yes	<code>{refresh}</code> → blacklisted (205)
GET	<code>/api/auth/me/</code>	yes	Current user
GET	<code>/api/conversations/</code>	yes	List: <code>{next, previous, results:[{id, title, created_at, updated_at, message_count}]}</code>
POST	<code>/api/conversations/</code>	yes	Create: <code>{title?, system_instruction?}</code>
GET	<code>/api/conversations/{id}/</code>	yes	One conversation with its <code>messages</code>
PATCH / PUT	<code>/api/conversations/{id}/</code>	yes	Rename / set <code>system_instruction</code>
DELETE	<code>/api/conversations/{id}/</code>	yes	Soft delete (204)
GET	<code>/api/conversations/{id}/messages/</code>	yes	Paged history, 50 per page, oldest first
POST	<code>/api/chat/</code>	yes	<code>{message, conversation_id?}</code> → SSE stream
GET	<code>/healthz/</code> , <code>/readyz/</code>	no	Liveness / readiness
GET	<code>/api/schema/</code> , <code>/api/docs/</code>	no	OpenAPI schema / interactive docs

Try it with curl

With the backend running locally, and using an account you've registered:

Terminal

```
# 1. Sign in and keep the access token in a shell variable
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/auth/login/ \
  -H 'Content-Type: application/json' \
  -d '{"email": "you@example.com", "password": "your-password"}' \
  | python3 -c 'import sys, json; print(json.load(sys.stdin)["access"])')

# 2. List your conversations
curl -s http://127.0.0.1:8000/api/conversations/ -H "Authorization: Bearer $TOKEN"

# 3. Send a message and watch the stream arrive (-N turns off curl's buffering)
curl -N -X POST http://127.0.0.1:8000/api/chat/ \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"message": "Say hello in three languages"}'
```

Step 3 prints `event: meta`, a run of `event: delta` frames and a final `event: done`, exactly the frames described in Chapter 3. Add `"conversation_id": "<id from meta>"` to the JSON to continue the same conversation.

Stream frames

Event	Data
meta	conversation_id, user_message_id, model, request_id, context_messages, dropped_messages
delta	text, repeated
done	message_id, conversation_id, finish_reason, title, usage { model, input_tokens, output_tokens, thinking_tokens, latency_ms, first_token_ms, used_fallback }
error	code, message, request_id
: heartbeat	(comment line, no data)

APPENDIX D

Design Decisions and Further Reading

The project records its important choices as ADRs in [docs/adr/](#). Each one is short and worth reading in full; here is the gist.

ADR	Decision	Why, in one breath
0001	JWT bearer tokens, not session cookies	The frontend is on a different domain; cookies across domains need <code>SameSite=None</code> plus CSRF tokens everywhere. Bearer tokens avoid that.
0002	Server-Sent Events for streaming	Only server-to-browser streaming is needed. WebSockets would add a second protocol and more infrastructure; plain chunked text can't say "finished" versus "cut off".
0003	UUID primary keys	IDs appear in URLs; sequential numbers are guessable and reveal volume.
0004	<code>thinking_level</code> , not <code>thinking_budget</code>	Gemini 3.x rejects <code>thinking_budget</code> with HTTP 400; live integration tests now guard the config.
0005	Tokens in <code>localStorage</code>	Keeps you signed in across reloads without cross-domain cookies. The XSS risk is limited by short token lives and by never rendering untrusted HTML.

The project's own documents

docs/ARCHITECTURE.md	Request path, layers, failure handling, data model
docs/SECURITY.md	Controls and known gaps
docs/RUNBOOK.md	Operating in production: incidents, cost, latency SQL
docs/DEPLOYMENT.md	The current Vercel + VPS setup in detail
docs/TESTING.md	Test layers and testing gotchas
docs/LLM_Powered_AI_Chatbot_PRD.pdf	The original requirements
backend/README.md , frontend/README.md	Per-half quick references

Learning more about the tools

Django tutorial	docs.djangoproject.com/en/stable/intro/tutorial01/
Django REST Framework tutorial	www.django-rest-framework.org/tutorial/quickstart/
React, "Learn React"	react.dev/learn
Vite guide	vite.dev/guide/
Tailwind CSS docs	tailwindcss.com/docs
Gemini API docs	ai.google.dev/gemini-api/docs
Server-Sent Events (MDN)	developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
HTTP status codes (MDN)	developer.mozilla.org/en-US/docs/Web/HTTP/Status
pytest / Vitest	docs.pytest.org · vitest.dev

Suggested next steps

1. Finish the five exercises in Chapter 14.
2. Read `backend/chat/views.py` and `frontend/src/hooks/useChat.js` top to bottom. Between them, they hold most of the interesting logic.
3. Pick an item from "Not implemented yet" in Chapter 16, such as password reset, and sketch how you'd build it.
4. Run `make integration` once to watch the live contract tests talk to Gemini.

End of guide. Happy building.